

POLITECNICO
MILANO 1863

DD by Hamidreza Khodadadi and
Mohammad Rashidkhan

# Design Document

| | |
|---|---|
| **Deliverable:** | DD |
| **Title:** | Design Document |
| **Authors:** | Mohammad Rashid Khan Mofrad and Hamidreza Khodadadi |
| **Version:** | 1.0 |
| **Date:** | 06-January-2026 |
| **Download page:** | https://github.com/hamidkhd/KhodadadiRashidkhan/blob/main/README.md |
| **Copyright:** | Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved |

# Contents

**Table of Contents** . . . . . **3**

**List of Figures** . . . . . **5**

**List of Tables** . . . . . **5**

**1 Introduction** . . . . . **6**
- 1.1 Purpose . . . . . 6
- 1.2 Scope . . . . . 6
- 1.3 Definitions, Acronyms, Abbreviations . . . . . 7
  - 1.3.1 Definitions . . . . . 7
  - 1.3.2 Acronyms . . . . . 8
  - 1.3.3 Abbreviations . . . . . 8
- 1.4 Revision History . . . . . 8
- 1.5 Reference Documents . . . . . 8
- 1.6 Document Structure . . . . . 9
- 1.7 Use of Generative AI Tools . . . . . 9
  - 1.7.1 Tool(s) Used . . . . . 9
  - 1.7.2 Inputs Provided . . . . . 9
  - 1.7.3 Outputs Obtained . . . . . 10
  - 1.7.4 Verification, Refinement, and Integration . . . . . 10

**2 Architectural Design** . . . . . **11**
- 2.1 Overview . . . . . 11
- 2.2 Component View . . . . . 12
- 2.3 Deployment View . . . . . 13
- 2.4 Runtime View . . . . . 15
- 2.5 Component Interfaces . . . . . 19
- 2.6 Selected Architectural Styles And Patterns . . . . . 24
  - 2.6.1 Architectural Styles . . . . . 24
  - 2.6.2 Design Patterns . . . . . 24
- 2.7 Other Design Decisions . . . . . 25
  - 2.7.1 Cloud Hosting . . . . . 25
  - 2.7.2 Security Measures . . . . . 25
  - 2.7.3 Availability and Scalability . . . . . 25
  - 2.7.4 Monitoring and Maintenance . . . . . 25

**3 User Interface Design** . . . . . **27**
- 3.1 Authentication . . . . . 27
- 3.2 Guest Overview . . . . . 28
- 3.3 Registered User Overview . . . . . 29
- 3.4 Manual Path . . . . . 33
- 3.5 Profile . . . . . 34
- 3.6 Feedback . . . . . 35
- 3.7 Administrative Overview . . . . . 36

**4 Requirements Traceability** . . . . . **37**
- 4.1 User Registration and Profile Management . . . . . 37
- 4.2 Trip Recording and History . . . . . 37
- 4.3 Weather Enrichment . . . . . 37

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

4.4 Manual Path and Obstacle Contribution . . . 38
4.5 Automated Detection and Issue Review . . . 38
4.6 Route Query and Map Visualization . . . 38
4.7 Administration . . . 39

**5 Implementation, Integration and Test Plan . . . 40**
5.1 Implementation Plan . . . 40
5.1.1 Phase 1 – Data Layer and Shared Foundations . . . 40
5.1.2 Phase 2 – Core Backend Services . . . 40
5.1.3 Phase 3 – Trip and Contribution Domain . . . 40
5.1.4 Phase 4 – Route Query and External Adapters . . . 41
5.1.5 Phase 5 – Administration and Clients . . . 41
5.1.6 Phase 6 – Hardening and Operational Readiness . . . 41
5.2 Integration Plan . . . 41
5.2.1 Stage 1 – Database + Repository/DAO Layer . . . 41
5.2.2 Stage 2 – Core Backend (Gateway + Auth + Profile) . . . 42
5.2.3 Stage 3 – Domain Services with Database . . . 42
5.2.4 Stage 4 – External Services via Adapters . . . 45
5.2.5 Stage 5 – End-to-End Client Integration . . . 45
5.3 Test Plan . . . 46
5.3.1 Unit Testing . . . 46
5.3.2 Integration Testing . . . 46
5.3.3 System Testing (End-to-End) . . . 47
5.3.4 Acceptance Testing . . . 47
5.3.5 Regression Testing . . . 47
5.3.6 Performance, Load, and Stress Testing . . . 47
5.3.7 Security Testing . . . 47
5.3.8 Deployment Testing . . . 47

**6 Effort Spent . . . 48**

**7 References . . . 49**

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# List of Figures


1 Component View . . . 13
2 Deployment View . . . 15
3 User authentication (login) . . . 16
4 Trip recording and local buffering (offline-first) . . . 16
5 Synchronization and trip closure (upload + persist) . . . 17
6 Trip enrichment with weather data . . . 18
7 Manual mode: create or update a path . . . 18
8 Route query: compute candidate paths and rank them . . . 19
9 Automated mode: detect issues and request user confirmation . . . 19
10 Sign Up page . . . 27
11 Login page . . . 27
12 Guest overview . . . 28
13 Filter paths . . . 28
14 Search routes . . . 29
15 Record Bike Path . . . 29
16 Record Bike Path Stats . . . 30
17 Manual report . . . 30
18 Detected issues status . . . 31
19 Review past detected issues . . . 31
20 Trip history . . . 32
21 Trip history details . . . 32
22 Create manual path . . . 33
23 Manual paths history . . . 33
24 Manual paths history details . . . 34
25 Registered user profile . . . 34
26 Registered user setting . . . 35
27 Registered user feedback . . . 35
28 Administrative overview . . . 36
29 Stage 1 Database baseline . . . 42
30 Stage 2 API Gateway skeleton . . . 42
31 Stage 3 Authentication and profile core . . . 42
32 Stage 4 Trip recording and upload . . . 43
33 Stage 5 Manual path and obstacle contribution . . . 43
34 Stage 6 Automated detection and issue review . . . 44
35 Stage 7 Route query and visualization (guest access) . . . 44
36 Stage 8 External providers via adapters (map + weather) . . . 45
37 Stage 9 Administration . . . 46


# List of Tables


1 Revision history of the DD. . . . 8
2 Traceability for registration and profile requirements. . . . 37
3 Traceability for trip recording and history requirements. . . . 37
4 Traceability for weather enrichment requirements. . . . 37
5 Traceability for manual contribution requirements. . . . 38
6 Traceability for automated detection and issue review requirements. . . . 38
7 Traceability for route query and visualization requirements. . . . 38
8 Traceability for administration requirements. . . . 39


Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 1 Introduction

This Design Document (DD) presents the design of *Best Bike Paths (BBP)*, a software system that supports cyclists in building and browsing an inventory of bike paths, and in exploiting community-contributed information to plan safer and more efficient routes. The DD provides the functional and architectural views needed to guide implementation, integration, and validation.

## 1.1 Purpose

Urban cyclists often struggle to identify *safe, comfortable, and efficient* routes because information about bike lanes, traffic conditions, road quality, and obstacles is frequently incomplete, outdated, or fragmented across different sources. At the same time, valuable data generated during rides (e.g., GPS traces and mobile-device sensor readings) is typically underutilized.

BBP addresses these issues by offering a platform to collect, maintain, and exploit a shared inventory of bike paths. Registered users can record trips, view trip statistics (optionally enriched with weather information), and contribute updates about path conditions and obstacles. Contributions can be provided either through manual input or through automated sensing, with a mandatory user confirmation/correction step before information becomes publishable.

The purpose of this DD is to translate the requirements captured in the RASD into a concrete and implementable design. It defines the overall architecture and the key design choices—including component decomposition and responsibilities, deployment assumptions, and runtime interactions—needed to deliver trip recording, publishable path information, and route search/visualization with scoring and ranking.

## 1.2 Scope

BBP supports both *data production* (collecting, validating, and publishing path information) and *data consumption* (searching, recommending, and visualizing bike paths). Registered users can record trips and store them as part of their cycling history; the system computes basic statistics (e.g., distance, duration, and average speed) and may enrich trips with meteorological data when an external weather service is available.

Registered users can also create and update bike-path information and decide whether it should be made publishable to the community. Data production is supported in two modes: (i) *manual mode*, where users select streets/segments and annotate them with status and obstacles, and (ii) *automated mode*, where BBP reconstructs the traveled path from GPS traces and detects potential issues using accelerometer/gyroscope signals. Since automated detection may generate false positives, users must confirm, correct, or reject detected issues before the information becomes visible to other users.

Any user (including non-registered users) can submit an origin–destination query and request BBP to visualize candidate paths on a map. When multiple alternatives exist, BBP ranks and presents them according to a computed score that reflects both the path conditions (e.g., status and obstacles) and its effectiveness in reaching the destination.

The platform supports the activities of three distinct actors:

- **Registered Cyclists:** record trips, access history and statistics, define and update paths and obstacles manually, enable automated detection, and confirm/correct/reject detected issues before publication.
- **Non-Registered Users (Guests):** browse publishable paths and submit route queries; they cannot record trips nor contribute or validate path information.
- **System Administrators:** operate and maintain BBP by monitoring availability, managing accounts, configuring external services (e.g., map/weather providers), and moderating or removing clearly incorrect or abusive information when needed.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 1.3 Definitions, Acronyms, Abbreviations

### 1.3.1 Definitions

- **Best Bike Paths (BBP):** The software system described in this document, aimed at supporting cyclists in recording trips, contributing validated information about bike paths, and obtaining route suggestions based on path quality.
- **Registered User (Cyclist):** A user with a BBP account who can record trips, access personal history and statistics, and contribute/validate path information.
- **Guest User:** A non-registered user who can browse publishable paths and request route suggestions, without recording trips or contributing/validating content.
- **Administrator:** A privileged actor responsible for platform operations (e.g., monitoring, configuration of external services, account management, and moderation of clearly incorrect or abusive content).
- **Trip:** A cycling activity recorded by BBP, typically represented by a time interval and an ordered sequence of geo-localized samples.
- **GPS Trace:** The sequence of geo-localized samples (latitude, longitude, timestamp) collected during a trip.
- **Bike Path (Path):** A route represented in BBP, either manually defined or derived from recorded trips, possibly enriched with community information (status and obstacles) and optionally marked as publishable.
- **Path Segment:** A portion of a path (e.g., a street segment) that can be independently annotated with status and obstacles.
- **Path Status:** A qualitative indicator associated with a path or segment to describe its usability/quality (e.g., optimal, medium, sufficient), used to support ranking and recommendations.
- **Obstacle:** A reported issue affecting cycling along a path segment (e.g., pothole, roadworks), inserted manually or proposed by automated sensing.
- **Detected Issue:** A candidate obstacle/anomaly inferred from sensor data analysis that requires user validation before it becomes publishable.
- **Publishable Information:** Path-related data (e.g., segments, status, obstacles) explicitly marked as shareable and therefore visible to other users.
- **Manual Mode:** Data production mode where the cyclist explicitly defines paths/segments and annotates them (status/obstacles).
- **Automated Mode:** Data production mode where BBP reconstructs the traveled path from GPS and identifies potential issues using inertial sensors; results are subject to user confirmation.
- **Confirmation:** The action by which a registered user validates, corrects, or rejects detected issues before they are published to the community.
- **Route Query:** A request specifying an origin and a destination (and optionally preferences) to obtain one or more recommended paths.
- **Candidate Path:** A path proposed by BBP in response to a route query and presented to the user (e.g., on a map), typically with a ranking score.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- **Score:** A numeric value used to rank candidate paths, computed from factors such as path status/obstacles and effectiveness in reaching the destination.
- **Safest Preference:** A routing strategy that prioritizes segments with "optimal" status and avoids all reported obstacles (e.g., potholes), even if it results in a longer travel distance.
- **Fastest Preference:** A routing strategy that minimizes the total travel distance and prioritizes paths where speed limits and road types allow for the highest average cycling speed.
- **Most Comfortable Preference:** A routing strategy focused on the cycling experience, prioritizing "optimal" road surface quality and avoiding segments requiring maintenance.
- **Balanced Preference:** The default routing strategy that provides a weighted trade-off between path safety, surface quality, and total travel distance.

### 1.3.2 Acronyms

- **BBP:** Best Bike Paths
- **DD:** Design Document
- **RASD:** Requirements Analysis and Specification Document
- **UML:** Unified Modeling Language

### 1.3.3 Abbreviations

- **UI:** User Interface
- **GPS:** Global Positioning System
- **API:** Application Programming Interface

## 1.4 Revision History

| Version | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | 2025-12-02 | Hamidreza Khodadadi, Mohammad Rashidkhan | Introduction section completed. |
| 0.3 | 2025-12-10 | Hamidreza Khodadadi, Mohammad Rashidkhan | Architectural Design section completed. |
| 0.5 | 2025-12-18 | Hamidreza Khodadadi, Mohammad Rashidkhan | User Interface section completed. |
| 0.7 | 2025-12-28 | Hamidreza Khodadadi, Mohammad Rashidkhan | Requirments Traceability section completed. |
| 0.8 | 2026-01-03 | Hamidreza Khodadadi, Mohammad Rashidkhan | Implementation section completed. |
| 1.0 | 2026-01-06 | Hamidreza Khodadadi, Mohammad Rashidkhan | Final review, formatting, and release. |

Table 1: Revision history of the DD.

## 1.5 Reference Documents

- Assignment specification (A.Y. 2025–2026): “Requirements and Design Documents” (RDD) and project description “Best Bike Paths”.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- Requirements Analysis and Specification Document (RASD) for BBP (group delivery).
- Slides from the Politecnico di Milano Software Engineering II course, available on the WeBeep platform.

## 1.6 Document Structure

This document is organized as follows:

- **Section 1 – Introduction:** introduces the system and this DD by presenting its purpose and scope, along with definitions, acronyms/abbreviations, revision history, reference documents, and the structure of the document.
- **Section 2 – Architectural Design:** presents the overall architecture of BBP, including the component decomposition, responsibilities and interactions, the deployment view, runtime behavior (e.g., sequence diagrams), and the main architectural patterns and design decisions.
- **Section 3 – User Interface Design:** describes the main user interfaces and interaction flows for the supported actors.
- **Section 4 – Requirements Traceability:** provides traceability between the RASD requirements/goals and the components, interfaces, and design elements presented in this DD.
- **Section 5 – Implementation, Integration and Test Plan:** outlines the planned implementation order, the integration strategy, and the testing approach adopted to validate BBP.
- **Section 6 – Effort Spent:** reports the effort spent by each group member for the different parts of the assignment.
- **Section 7 – References:** lists bibliographic references and any additional material cited in this DD.

## 1.7 Use of Generative AI Tools

To support the preparation and refinement of this Design Document, we used Generative AI tools during the development process. This subsection reports the tool(s) used, the inputs provided, the outputs obtained, and the verification steps adopted to ensure correctness and alignment with the project requirements.

### 1.7.1 Tool(s) Used

- ChatGPT (OpenAI)

### 1.7.2 Inputs Provided

1. Requests to improve clarity, coherence, and cohesion across the document, including the identification of inconsistencies and the enforcement of consistent terminology.
2. Requests to support the production of UML diagram specifications in PlantUML (e.g., component, deployment, and sequence diagrams).
3. Requests to suggest and refine architectural choices, component responsibilities, and interactions to ensure alignment with the scope and with the requirements defined in the RASD.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

### 1.7.3 Outputs Obtained

- Textual revisions to improve readability and technical precision, and to maintain consistent terminology throughout the document.
- PlantUML diagram code and recommendations to support the production of UML diagrams (e.g., component, deployment, and sequence diagrams).
- Suggestions for architectural decomposition and for mapping requirements to components, interfaces, and responsibilities.

### 1.7.4 Verification, Refinement, and Integration

- All generated suggestions were reviewed to ensure consistency with the assignment constraints and the requirements stated in the RASD.
- For PlantUML outputs, the generated code was executed to verify that diagrams compiled and rendered correctly.
- When issues were identified (e.g., incorrect assumptions, missing details, inconsistent terminology, or PlantUML syntax errors), the content was manually refined until a correct and project-consistent result was obtained.
- Only verified and project-consistent outputs were incorporated into the final version of the document.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 2 Architectural Design

## 2.1 Overview

The architectural design of the Best Bike Paths (BBP) system follows a modular and layered architecture to support trip recording, path contribution, and route query execution through a scalable and maintainable solution. The system serves three categories of users: registered cyclists, guest users, and system administrators. Registered cyclists can record trips, submit path information in manual mode, and review issues detected in automated mode. Guest users can only execute route queries and visualize candidate paths. System administrators manage accounts and configuration parameters, including connections to external services.

**Three-tier architecture for modularity and maintainability**
To separate responsibilities and to enable independent evolution of user interfaces, business logic, and data management, BBP adopts a three-tier architecture consisting of a Presentation Layer, an Application Layer, and a Data Layer. Each layer exposes well-defined interfaces and communicates through explicit protocols.

- **Presentation Layer:** This layer includes the mobile client and the web client used by cyclists and guest users, and the administrator web client used by system administrators. The Presentation Layer provides the user interface for trip recording, manual path submission, detected issue review, and route query execution. It interacts with the Application Layer through the BBP REST API, and it is responsible for handling device permissions, sensor access, and offline buffering on the mobile device.

- **Application Layer:** This layer implements the backend logic through a set of modular services. It exposes a REST API for authentication, trip upload and retrieval, path creation and modification, detected issue management, and route query handling. The Application Layer also integrates external services through dedicated adapter components to isolate third-party dependencies and to support future replacement of the map provider and the weather provider.

- **Data Layer:** This layer provides persistent storage for users, trips, paths, obstacles, and configuration data through a DBMS. All access to persistent storage is mediated by the Application Layer. The Data Layer is designed to preserve data consistency and to prevent partial records in the presence of failures.

**Interaction Between Layers**
The Presentation Layer interacts with the Application Layer by invoking REST API endpoints using HTTP or HTTPS. The Application Layer validates requests, executes the related business logic, and persists data through the Data Layer. When network connectivity is not available during a trip, the mobile client stores trip data locally and synchronizes the buffered data once connectivity is restored. The Application Layer is responsible for resolving conflicts arising from concurrent updates.

**Alignment with Application Goals**
The architecture is consistent with the required communication and security constraints: data exchanged through the public API uses JSON with UTF-8 encoding, and all communication carrying authentication credentials or personal data uses HTTPS with TLS. The separation into modular services supports availability and scalability goals for core operations such as trip storage and route query execution. The use of adapters for external services ensures that temporary failures of the map provider or weather provider do not prevent core operations, and that the system remains evolvable as external APIs change.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 2.2 Component View

The component view describes the logical decomposition of BBP into components and the responsibilities of each component within the three-tier architecture. Components are grouped into Presentation Layer, Application Layer, and Data Layer. Interactions between the Presentation Layer and the Application Layer occur through the BBP REST API, which supports user authentication, trip upload and history retrieval, path creation and modification, route query execution, and management of detected issues.

**Presentation Layer**

- **Mobile Client**: User interface for registered cyclists. It provides trip recording, trip history browsing, manual mode interaction, automated mode activation, and detected issue review. During trip recording it periodically acquires GPS samples and, in automated mode, motion sensor data, and buffers the collected data locally when connectivity is not available.
- **Web Client**: User interface for route query execution and visualization. It supports origin–destination query composition and map visualization of candidate paths for both registered cyclists and guest users.
- **Administrator Web Client**: User interface for system administrators. It supports monitoring system status, viewing aggregated statistics, managing user accounts, and configuring connections to external services.

**Application Layer**

- **API Gateway**: Single entry point exposing the BBP REST API to the Presentation Layer. It routes incoming requests to the appropriate internal services and enforces uniform request validation and authentication checks.
- **Authentication Service**: Handles user registration and authentication, and issues authentication tokens used to access features reserved to registered cyclists.
- **User Profile Service**: Manages user profile data and privacy settings, including default publication preferences for contributed information.
- **Trip Service**: Manages trip lifecycle operations, including trip upload, trip reconstruction, storage, and trip history retrieval. It computes trip statistics and associates them with stored trips.
- **Weather Adapter**: Integrates the external weather service to enrich recorded trips with meteorological information when the service is reachable. The adapter isolates third-party dependencies from core services.
- **Path Service**: Manages manual mode operations for bike paths, including path definition as a sequence of connected segments, path status assignment, obstacle annotation, and publication of path information.
- **Issue Detection Service**: Supports automated mode by processing motion sensor data collected during trip recording to produce detected issues associated with trip and path segments.
- **Issue Review Service**: Manages the confirmation phase by presenting detected issues and applying user actions (confirm, correct, reject). Confirmed issues are converted into obstacles and can be made publishable according to user settings.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- **Route Query Service**: Executes route queries by retrieving candidate paths from the inventory. The service calculates path scores by applying specific algorithmic weights to distance, status, and obstacles based on the user's selected preference: Safest, Fastest, Most Comfortable, or Balanced. This allows the system to rank candidate paths according to different user-defined priorities.
- **Map Adapter**: Integrates the external map service to display maps, convert addresses to coordinates, and obtain base street network information used to define and evaluate paths. The adapter isolates third-party dependencies from core services.
- **Notification Service**: Sends notifications to the mobile client when required, for example to confirm trip persistence or to remind the user to review detected issues, using platform-specific notification mechanisms.
- **Administration Service**: Supports administrator operations such as account management and configuration of external service connections.

**Data Layer**

- **Database**: Central persistent storage for users, trips, paths, obstacles, and configuration data. All access to the database is mediated by the Application Layer, and clients do not interact with the database directly.

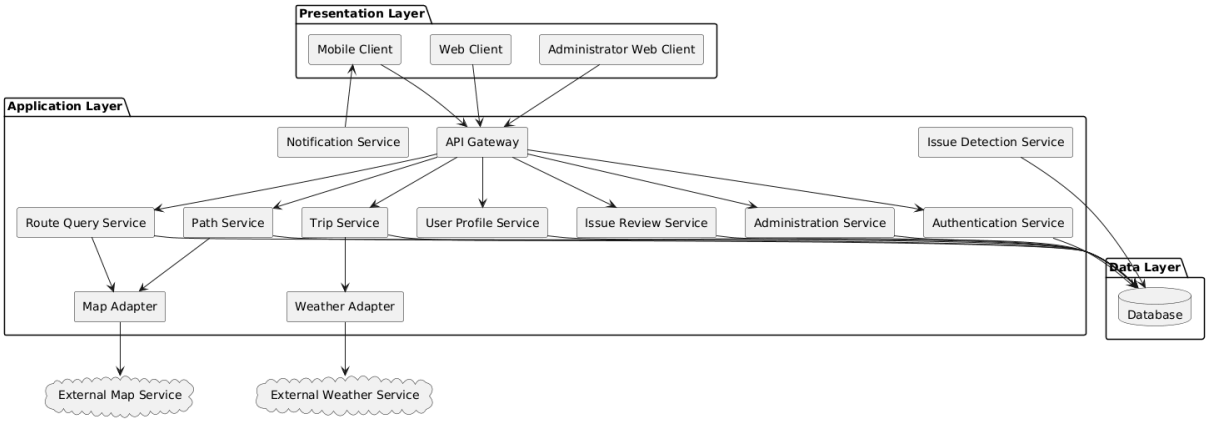



Figure 1: Component View

## 2.3 Deployment View

The deployment view describes the execution environment of BBP and the physical infrastructure on which the system operates. The deployment architecture is designed to support scalability, availability, and secure communication between clients, backend services, persistent storage, and external providers. All client–server interactions occur through the BBP REST API using JSON encoded in UTF-8. Communication that transports credentials or personal data uses HTTPS with TLS.
BBP is deployed in a cloud environment and consists of the following nodes:

- **User Device (Mobile) :** A smartphone hosting the Mobile Client. It accesses device sensors (GPS and, in automated mode, motion sensors) and stores buffered trip data locally when connectivity is not available. Buffered data is synchronized with the backend when connectivity is restored.
- **User Device (Web) :** A device hosting a web browser that runs the Web Client. It is used to submit route queries and visualize candidate paths.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- **Perimeter Security Node :** A firewall or equivalent cloud security service that restricts inbound traffic to the public API endpoints and enforces network-level policies. This node reduces exposure of internal services and supports secure separation between public-facing endpoints and internal resources.

- **Load Balancer :** Distributes incoming HTTPS requests across multiple instances of the Application Server node. It supports high availability and elastic scaling under variable load.

- **Application Server Node :** Hosts the Application Layer components, including the API Gateway and the internal services responsible for authentication, trip management, path management, issue management, and route query execution. These services access persistent storage through the Database node and integrate third-party providers through adapter components.

- **Database Node :** Hosts the DBMS that stores user profiles, trips, paths, obstacles, and configuration data. The database is not directly accessible from client devices and is reachable only from the Application Server node.

- **External Map Service :** Third-party provider used for map visualization and related operations. It is accessed only through the Map Adapter component to isolate provider dependencies and to enable substitution if needed.

- **External Weather Service :** Third-party provider used to enrich recorded trips with weather information when available. It is accessed only through the Weather Adapter component to isolate provider dependencies.

This deployment structure maintains a clear separation between public clients and internal resources, supports the offline-first behavior required during trip recording, and ensures that changes in external provider APIs remain confined to adapter components.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

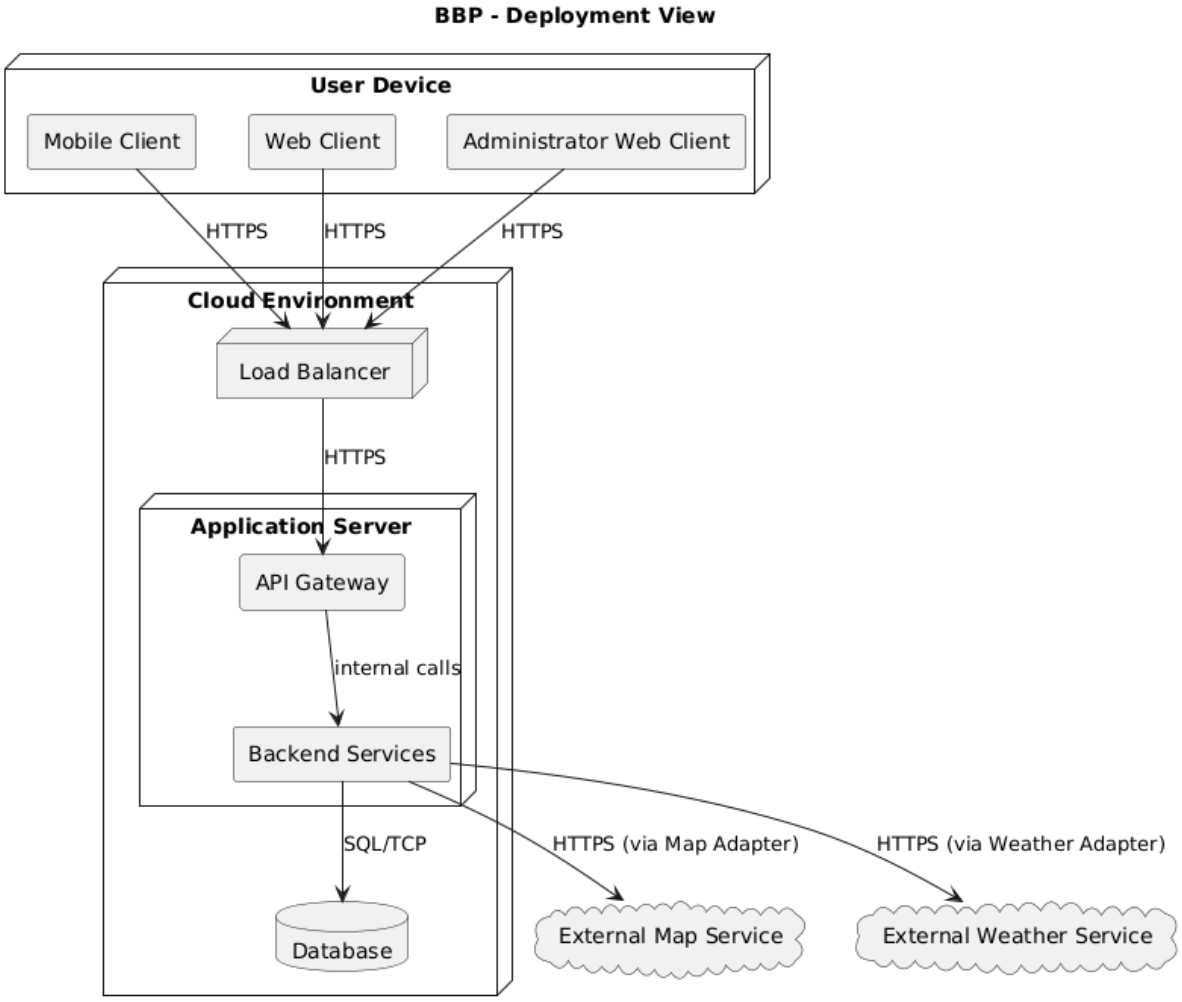



Figure 2: Deployment View

## 2.4 Runtime View

The runtime view describes how BBP components collaborate to execute the most important system operations. The interactions are represented through sequence diagrams. Client-side elements such as the mobile operating system and the web browser are shown only to clarify sensor acquisition, local buffering, and user interaction, and they are not considered system components. All client–server interactions are executed through the BBP REST API exposed by the API Gateway. The diagrams focus on: (i) trip recording and synchronization, (ii) manual path contribution, (iii) route query execution, and (iv) issue detection and user confirmation, because these operations correspond to the core use cases of BBP.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

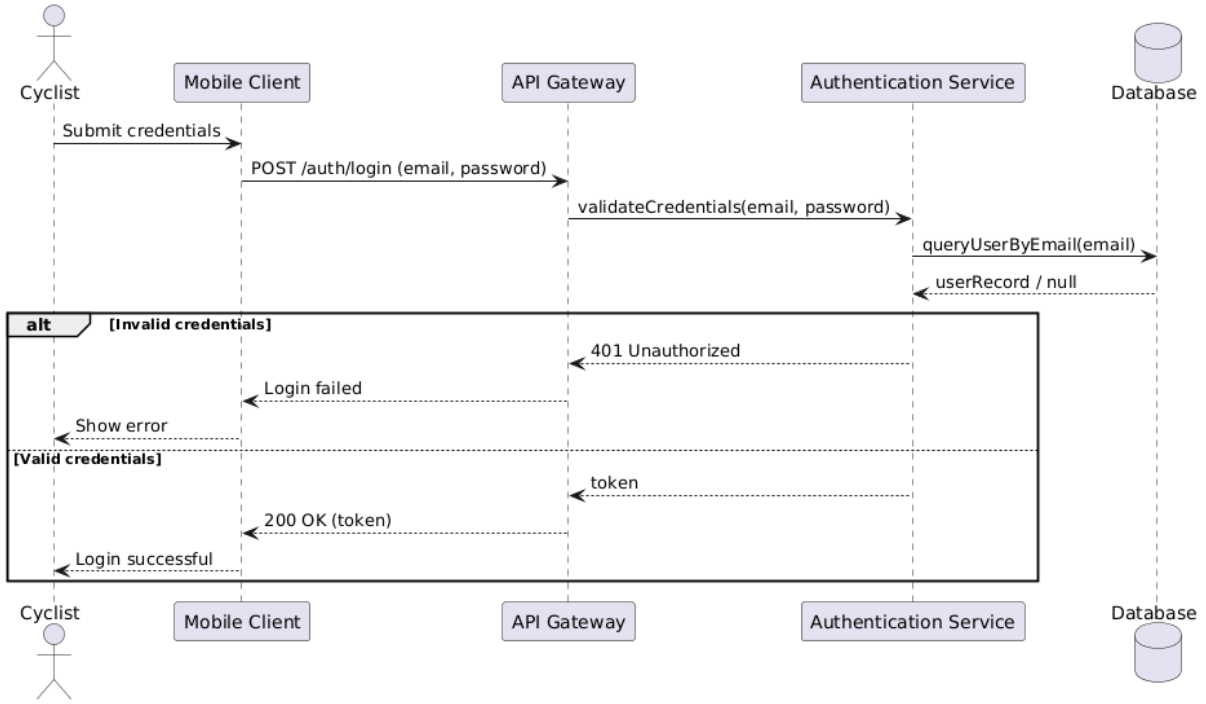



Figure 3: User authentication (login)

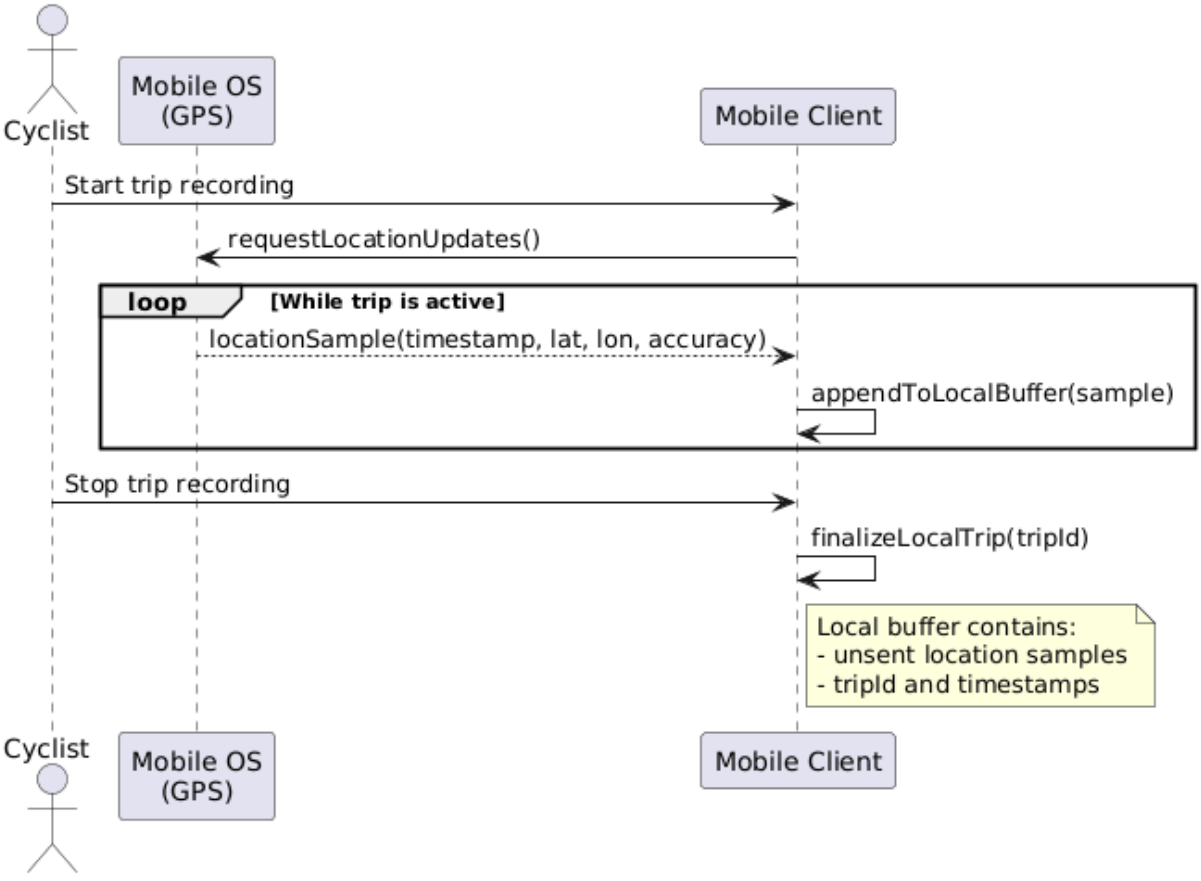



Figure 4: Trip recording and local buffering (offline-first)

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

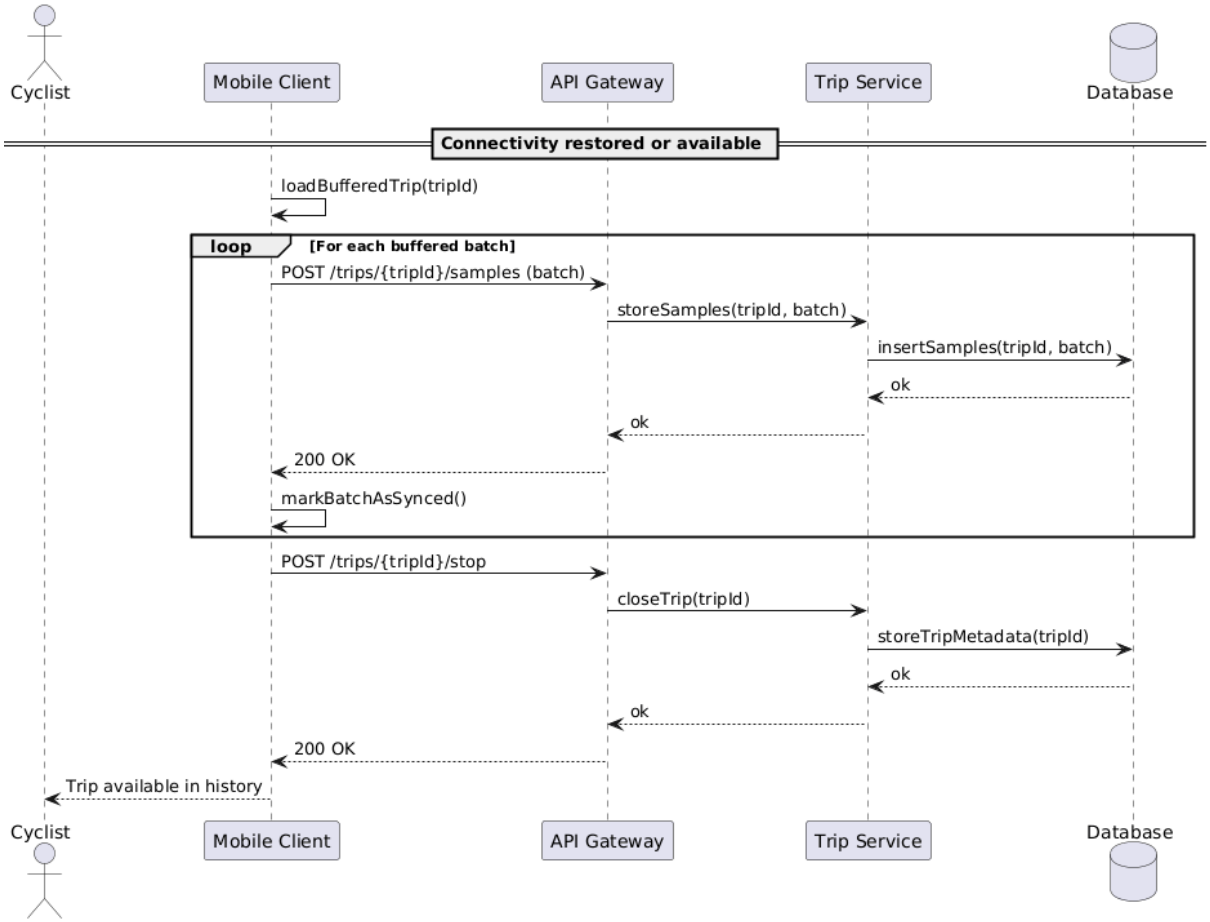



Figure 5: Synchronization and trip closure (upload + persist)

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

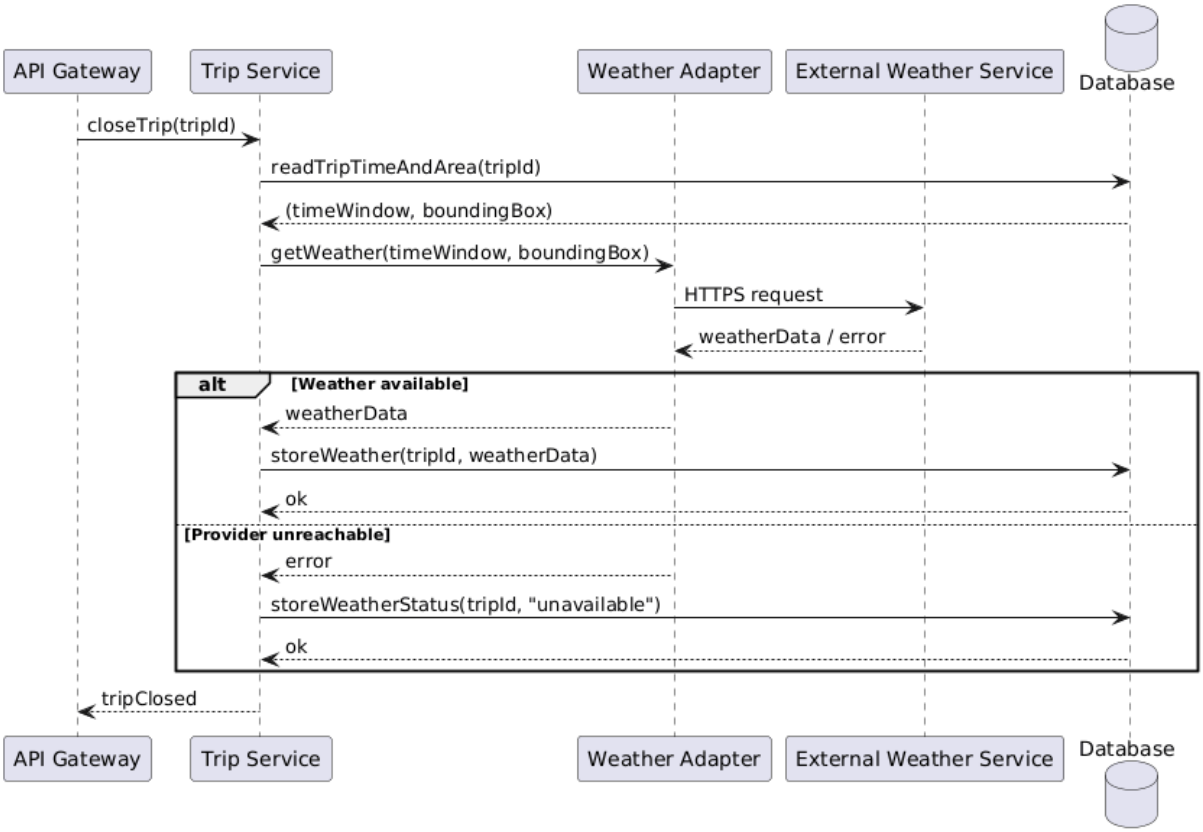



Figure 6: Trip enrichment with weather data

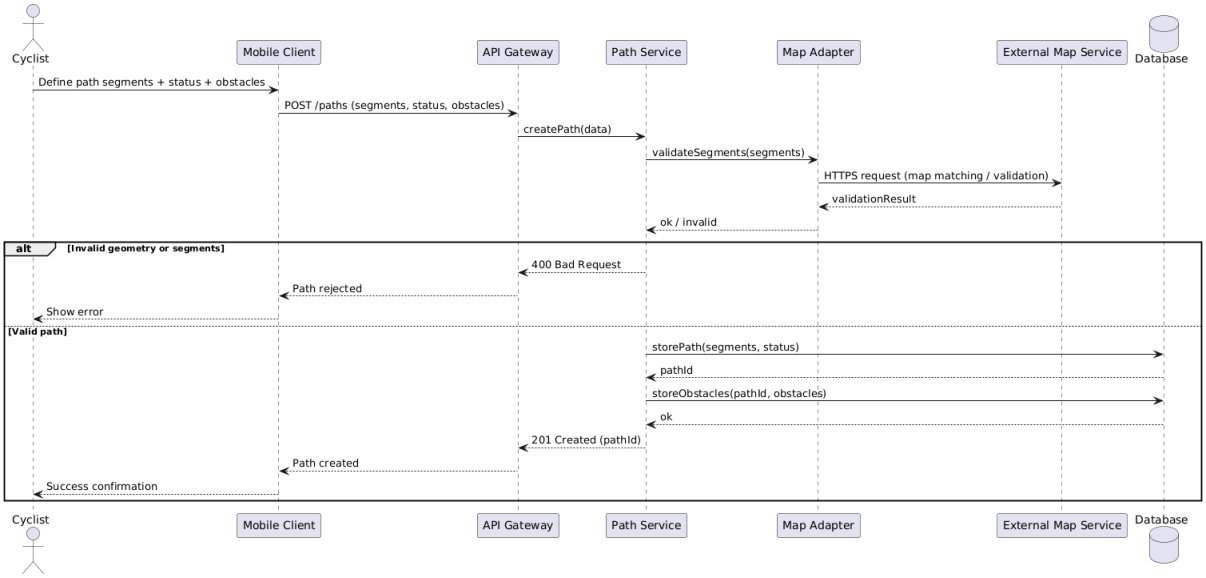



Figure 7: Manual mode: create or update a path

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

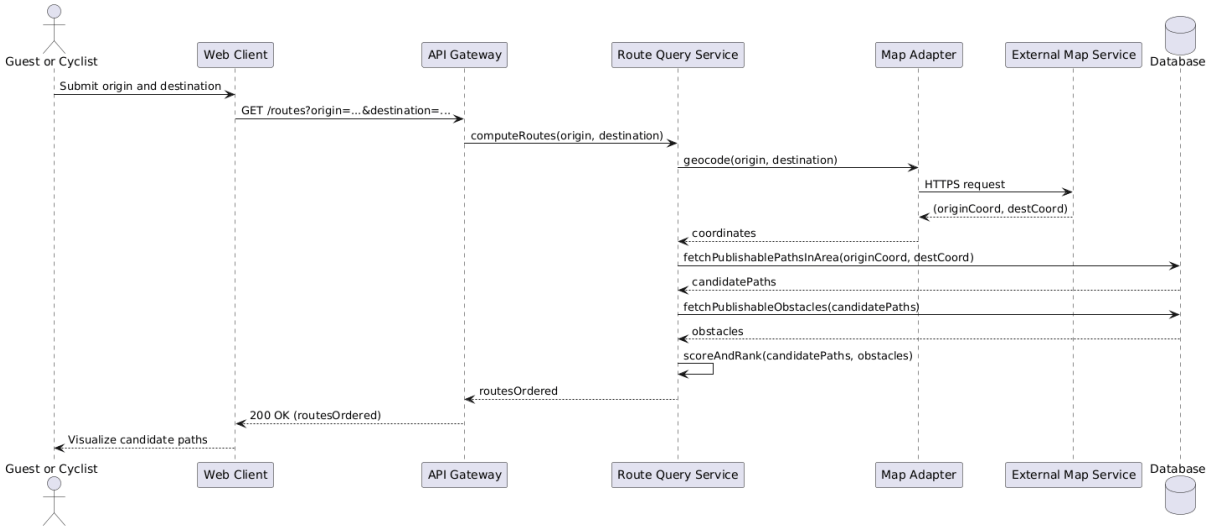



Figure 8: Route query: compute candidate paths and rank them

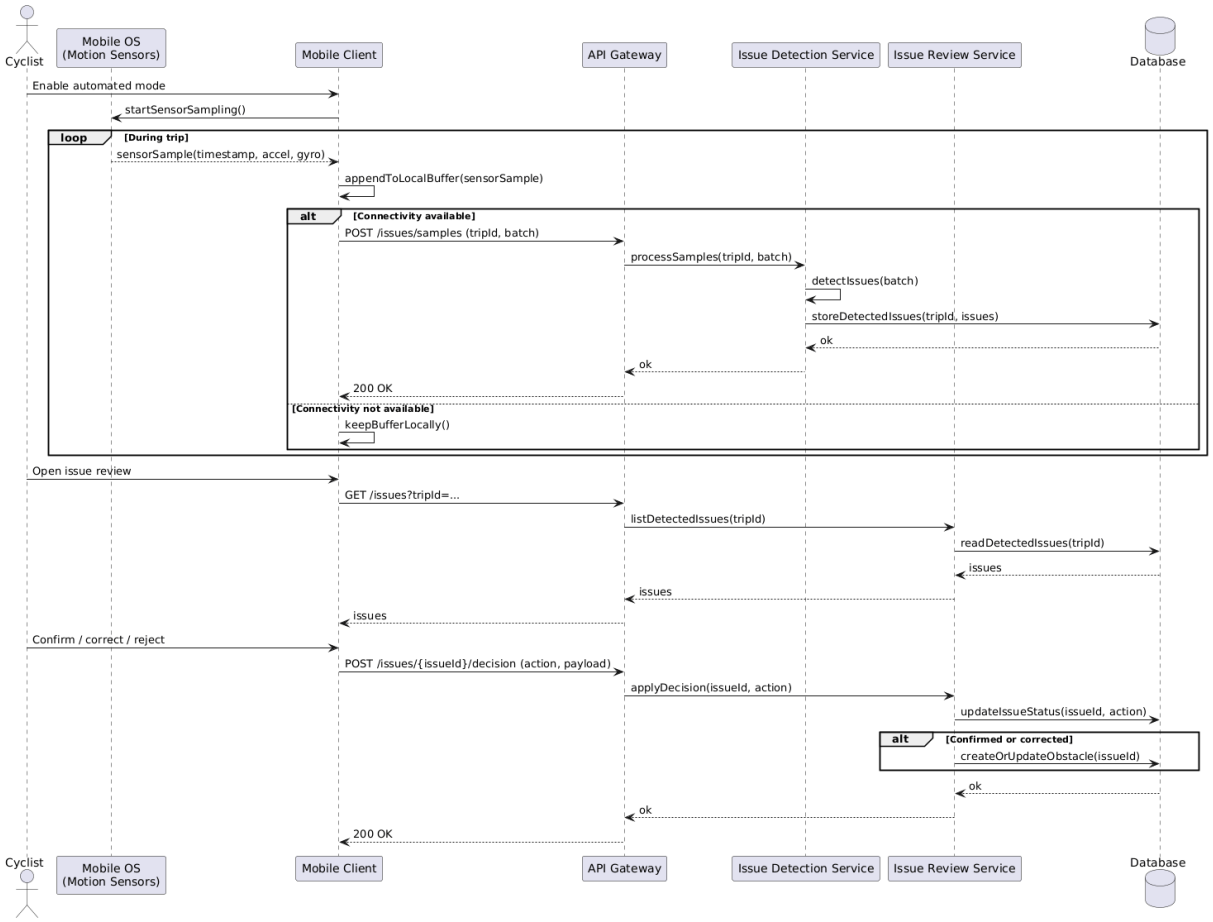



Figure 9: Automated mode: detect issues and request user confirmation

## 2.5 Component Interfaces

This section describes the public interfaces exposed by BBP components. Interfaces are specified at an abstract level to clarify how components interact at runtime. The Presentation Layer interacts with the

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

Application Layer exclusively through the REST API exposed by the API Gateway.

**Mobile Client**

- **startTripRecording(mode: TripMode):** Starts trip acquisition in manual mode or automated mode.
- **stopTripRecording(tripId: UUID):** Stops acquisition and finalizes the trip locally.
- **bufferSample(tripId: UUID, sample: LocationSample):** Stores a location sample locally when the network is unavailable.
- **bufferSensorSample(tripId: UUID, sample: SensorSample):** Stores a sensor sample locally in automated mode.
- **syncTrip(tripId: UUID):** Uploads buffered trip batches and invokes trip closure through the REST API.
- **reviewDetectedIssues(tripId: UUID):** Requests the list of detected issues and supports confirmation actions.

**Web Client**

- **submitRouteQuery(origin: Location, destination: Location, preferences: QueryPreferences):** Submits a route query request.
- **displayCandidatePaths(routes: RouteList):** Visualizes the returned candidate paths and their scores.
- **viewPathDetails(pathId: UUID):** Requests detailed information about a specific path.

**Administrator Web Client**

- **viewSystemMetrics():** Requests aggregated system metrics and status.
- **manageUserAccount(userId: UUID, action: AdminAction):** Triggers administrative actions on user accounts.
- **updateExternalServiceConfig(config: ExternalServiceConfig):** Updates configuration used by adapter components.

**Application Layer interfaces**

**API Gateway (BBP REST API)**

**Authentication**

- **POST /auth/register**(payload: RegistrationData) → (userId, token)
- **POST /auth/login**(email, password) → (token)
- **POST /auth/logout**() → status

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

**User profiles**

- **GET /users/me**() → UserProfile
- **PATCH /users/me**(ProfileUpdate) → UserProfile

**Trips**

- **POST /trips**(TripCreate) → (tripId)
- **POST /trips/{tripId}/samples**(SampleBatch) → status
- **POST /trips/{tripId}/stop**() → status
- **GET /trips/{tripId}**() → TripDetails
- **GET /trips**(filters) → TripList

**Paths (manual mode)**

- **POST /paths**(PathCreate) → (pathId)
- **PATCH /paths/{pathId}**(PathUpdate) → status
- **POST /paths/{pathId}/obstacles**(ObstacleCreate) → (obstacleId)
- **GET /paths/{pathId}**() → PathDetails

**Detected issues (automated mode)**

- **POST /issues/samples**(IssueSampleBatch) → status
- **GET /issues**(tripId) → IssueList
- **POST /issues/{issueId}/decision**(IssueDecision) → status

**Route query**

- **GET /routes**(origin, destination, preferences) → RouteList

**Administration**

- **GET /admin/metrics**() → Metrics
- **POST /admin/users/{userId}/action**(AdminAction) → status
- **PATCH /admin/config/external-services**(ExternalServiceConfig) → status

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

**Authentication Service**

- **registerUser(data: RegistrationData):** Creates a new user account and returns an authentication token.
- **login(email: Email, password: Password):** Validates credentials and returns an authentication token.
- **validateToken(token: Token):** Validates the token and returns authorization context.

**User Profile Service**

- **getProfile(userId: UUID):** Returns the profile data for the specified user.
- **updateProfile(userId: UUID, update: ProfileUpdate):** Updates profile fields and privacy settings.

**Trip Service**

- **createTrip(userId: UUID, tripCreate: TripCreate):** Creates a trip container and returns a trip identifier.
- **storeSamples(tripId: UUID, batch: SampleBatch):** Persists a batch of location samples.
- **closeTrip(tripId: UUID):** Finalizes the trip and computes derived attributes.
- **getTrip(tripId: UUID):** Returns trip details for visualization.
- **listTrips(userId: UUID, filters: TripFilters):** Returns the trip history.

**Weather Adapter**

- **getWeather(timeWindow: TimeWindow, area: GeoArea):** Queries the external weather service and returns weather data or an error.

**Path Service**

- **createPath(userId: UUID, pathCreate: PathCreate):** Persists a new path contributed in manual mode.
- **updatePath(pathId: UUID, pathUpdate: PathUpdate):** Applies updates to an existing path.
- **addObstacle(pathId: UUID, obstacleCreate: ObstacleCreate):** Stores an obstacle associated with a path.
- **getPath(pathId: UUID):** Returns detailed information for a path.

**Issue Detection Service**

- **processSamples(tripId: UUID, batch: IssueSampleBatch):** Processes sensor samples and produces detected issues.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

**Issue Review Service**

- **listDetectedIssues(tripId: UUID):** Returns detected issues that require user review. RASDv1
- **applyDecision(issueId: UUID, decision: IssueDecision):** Updates the issue status and, when applicable, creates or updates obstacles.

**Route Query Service**

- **computeRoutes(origin: Location, destination: Location, preferences: QueryPreferences):** Retrieves candidate paths, computes scores using path status and publishable obstacles, and returns ranked results.

**Map Adapter**

- **geocode(input: AddressOrPOI):** Translates user input to coordinates through the external map service.
- **validateSegments(segments: SegmentList):** Validates contributed path segments through the external map service.

**Notification Service**

- **sendNotification(userId: UUID, notification: Notification):** Sends a notification to a user through the mobile platform channel.

**Administration Service**

- **getMetrics():** Returns aggregated metrics for administrative monitoring.
- **applyUserAction(userId: UUID, action: AdminAction):** Applies an administrative action to a user account.
- **updateExternalServiceConfig(config: ExternalServiceConfig):** Stores configuration required by adapter components.

**Data Layer interfaces**

**Database**
**The database is accessed only by Application Layer services.**

- **insert(entity: EntityName, record: Record):** Inserts a record.
- **update(entity: EntityName, key: Key, record: Record):** Updates a record.
- **select(entity: EntityName, filter: Filter):** Retrieves records matching the filter.
- **delete(entity: EntityName, key: Key):** Deletes a record.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 2.6 Selected Architectural Styles And Patterns

This section motivates the architectural styles and design patterns adopted for BBP. The selected solutions aim to keep the system modular, maintainable, and evolvable, while supporting the required operational constraints, including offline trip recording and integration with external providers.

### 2.6.1 Architectural Styles

**Three-tier architecture.**
BBP adopts a three-tier architecture composed of Presentation Layer, Application Layer, and Data Layer. The Presentation Layer includes the Mobile Client, the Web Client, and the Administrator Web Client. The Application Layer provides the business logic through modular services exposed through the REST API of the API Gateway. The Data Layer stores persistent data through a DBMS and is accessed only by Application Layer services. This organization supports separation of responsibilities and limits the propagation of changes across layers. The communication between Presentation Layer and Application Layer is performed through REST calls using JSON with UTF-8 encoding, and HTTPS with TLS is used when credentials or personal data are transmitted.

**Modular service decomposition .**
The Application Layer is decomposed into modular services aligned with distinct responsibilities, including authentication, user profile management, trip management, path management, issue management, and route query execution. This decomposition improves maintainability by localizing changes and enables independent evolution of services. This decision is consistent with the requirement to structure the system as a set of modular services.

**Offline-first client behavior.**
BBP adopts offline-first behavior for trip recording on the Mobile Client. When connectivity is unavailable, the Mobile Client buffers trip samples locally and synchronizes them when connectivity is restored. This style ensures that temporary connectivity failures do not prevent trip recording and persistence.

### 2.6.2 Design Patterns

1. **Adapter pattern (external providers)**:
   BBP uses the Adapter pattern to integrate external providers. The Map Adapter encapsulates all interactions with the external map service. The Weather Adapter encapsulates all interactions with the external weather service. This pattern isolates external provider APIs from the core services and supports substitution of the provider with limited changes confined to the adapter components. This decision is consistent with the constraint of isolating the external map service and the external weather service behind interfaces.

2. **Facade pattern**:
   BBP uses the Facade pattern through the API Gateway. The API Gateway exposes a single REST API to the Presentation Layer and delegates requests to internal services. This reduces coupling between clients and internal services and centralizes cross-cutting concerns such as authentication checks and request validation.

3. **Repository pattern**:
   BBP applies the Repository pattern within Application Layer services to separate business logic from data access operations. Each service accesses persistent data through repository-like components that encapsulate query and persistence logic. This supports testability and allows the persistence strategy to evolve without changing service interfaces. The database remains inaccessible to clients and is reachable only through Application Layer services.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 2.7 Other Design Decisions

This section documents additional design decisions that affect the deployment and operation of BBP, including hosting, security, availability and scalability, and monitoring and maintenance.

### 2.7.1 Cloud Hosting

BBP is deployed in a cloud environment to support scalable execution and managed infrastructure. The deployment includes a load balancer that distributes incoming requests across multiple Application Server instances. The database is deployed as a managed DBMS service to support automated backups and operational reliability.

### 2.7.2 Security Measures

- **Authentication enforcement:** BBP restricts trip upload, manual path contribution, and issue review to authenticated users. Guest users are restricted to route query execution and visualization.
- **Transport security:** BBP uses HTTPS with TLS for communication carrying credentials or personal data. Data exchanged through the REST API is encoded in JSON using UTF-8.
- **Network protection:** BBP applies perimeter protection through a firewall or equivalent cloud security service that restricts inbound traffic to the public API endpoints and prevents direct access to internal resources.
- **Credential storage:** Passwords are stored only in hashed form and are never stored or transmitted in plain text.

### 2.7.3 Availability and Scalability

- **Load balancing and horizontal scaling:** BBP increases throughput by running multiple instances of Application Layer services behind the load balancer. This supports concurrency for route queries and trip uploads.
- **External provider isolation:** BBP isolates failures of the external map service and external weather service through adapter components. Core services remain operational when an external provider is unavailable, and the system degrades functionality without violating consistency of stored data.
- **Offline-first trip recording:** BBP ensures that trip recording continues when connectivity is unavailable by buffering samples locally and synchronizing them when connectivity is restored.

### 2.7.4 Monitoring and Maintenance

- **Logging:** Application Layer services produce centralized logs that support fault diagnosis and traceability across services.
- **Health Checks:** Each service exposes a health endpoint used by the orchestration platform to detect failures and replace unhealthy instances.
- **Metrics:** BBP collects metrics such as request rate, error rate, request latency, database response time, and external provider availability.
- **Backup and Recovery:** The DBMS is configured with periodic backups and recovery procedures to prevent data loss.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- **Continuous integration and deployment:** Automated pipelines execute tests and checks before deployment and support controlled releases.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 3 User Interface Design

## 3.1 Authentication

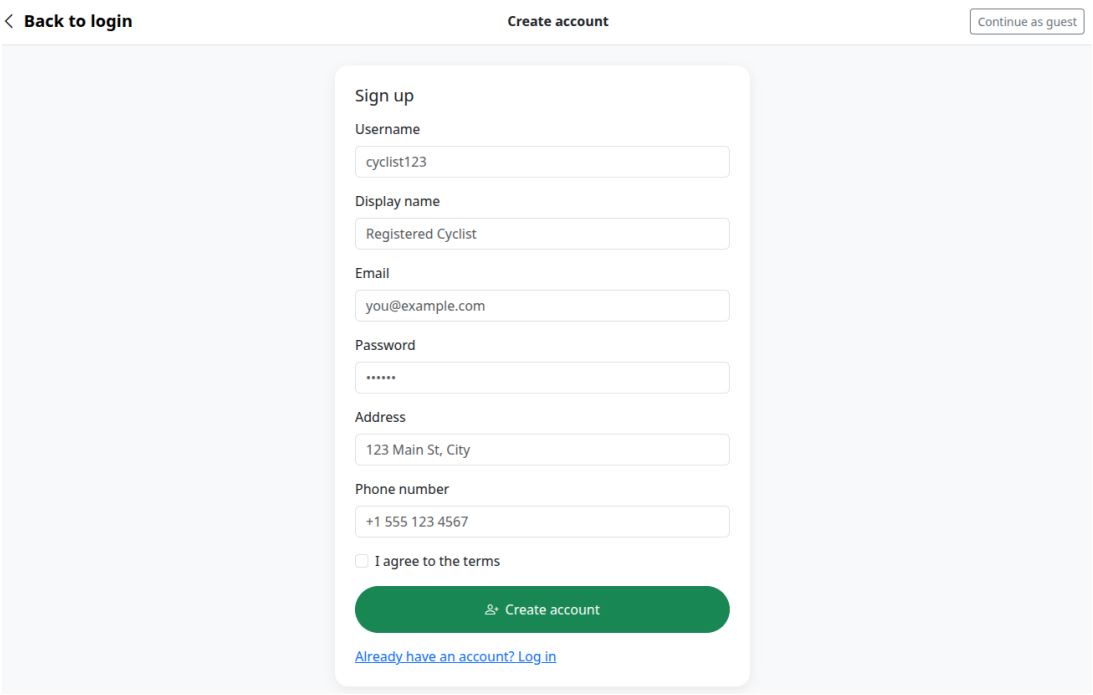



Figure 10: Sign Up page

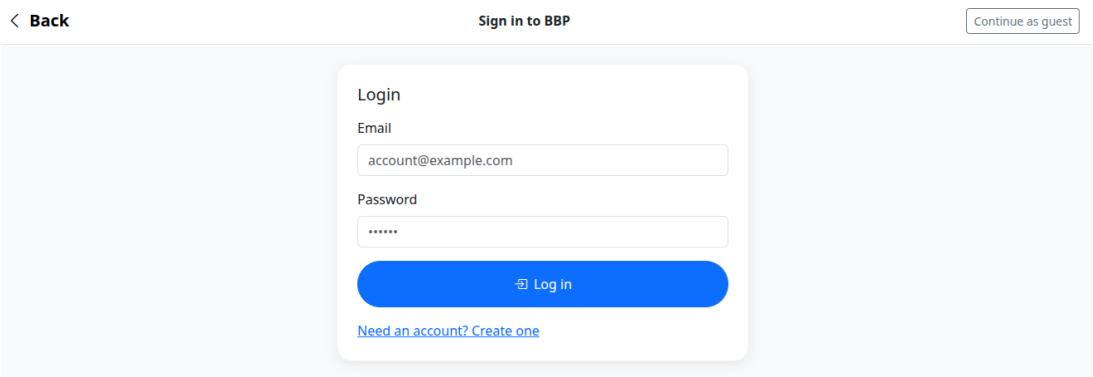



Figure 11: Login page

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 3.2 Guest Overview

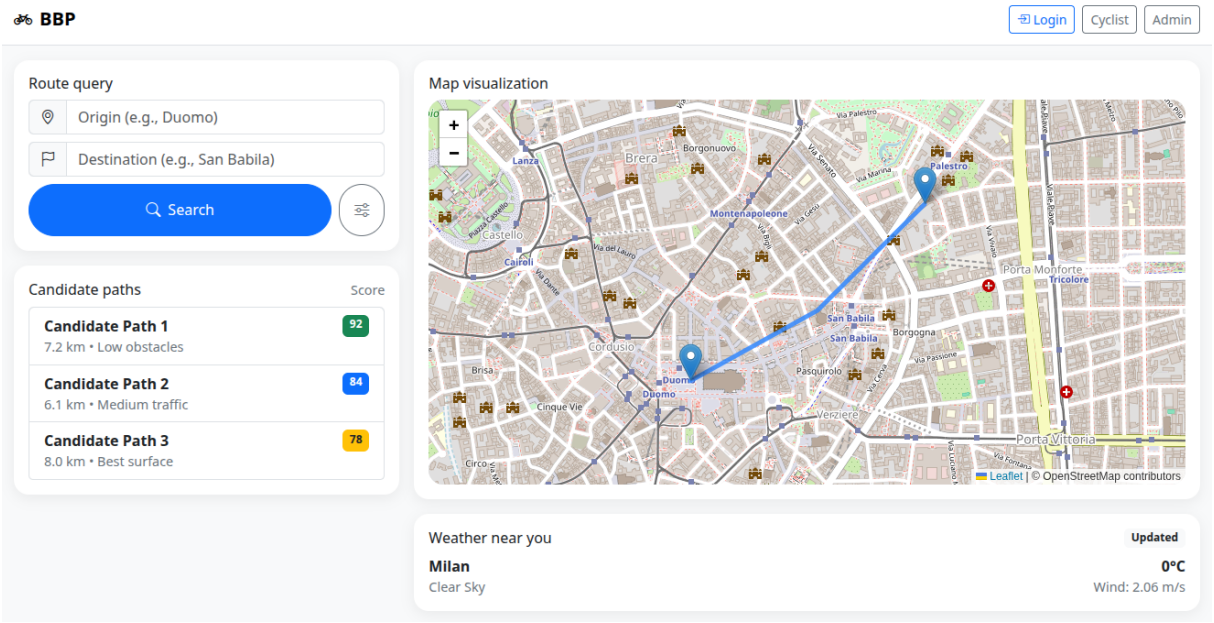



Figure 12: Guest overview

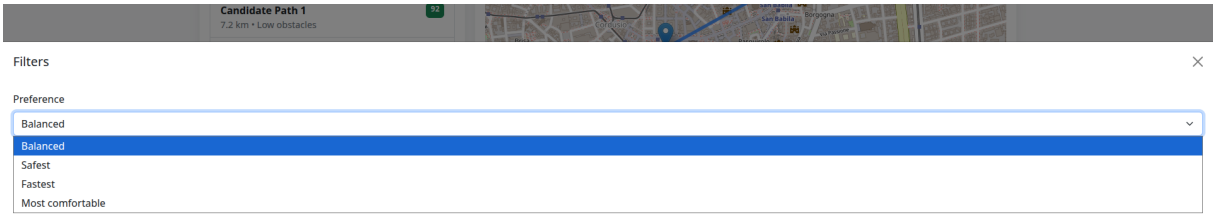



Figure 13: Filter paths

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 3.3 Registered User Overview

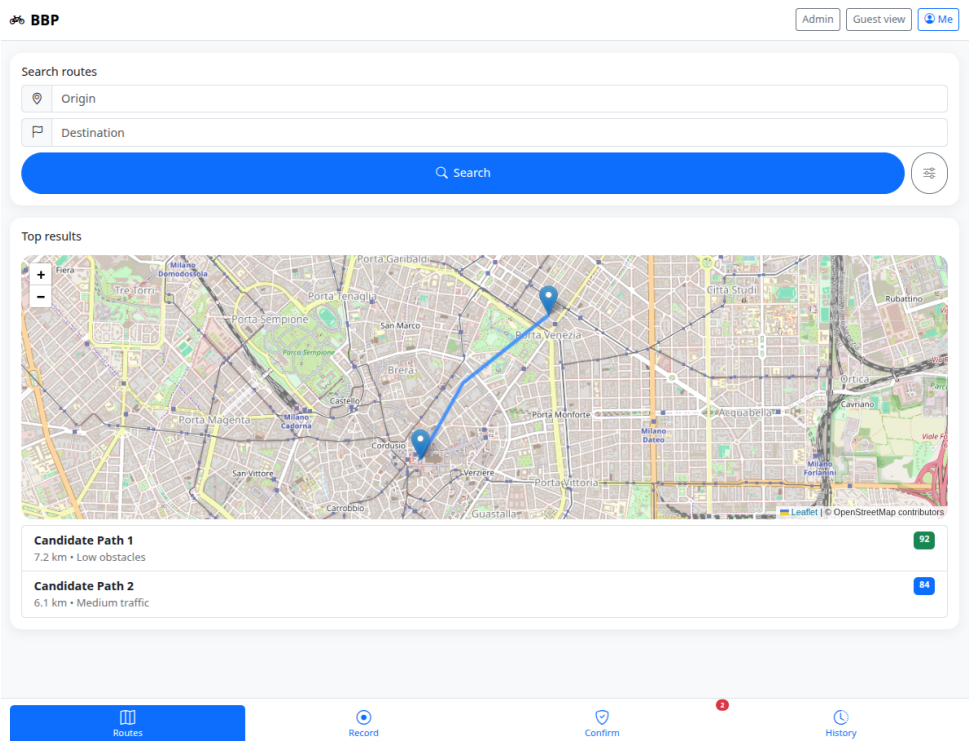



Figure 14: Search routes

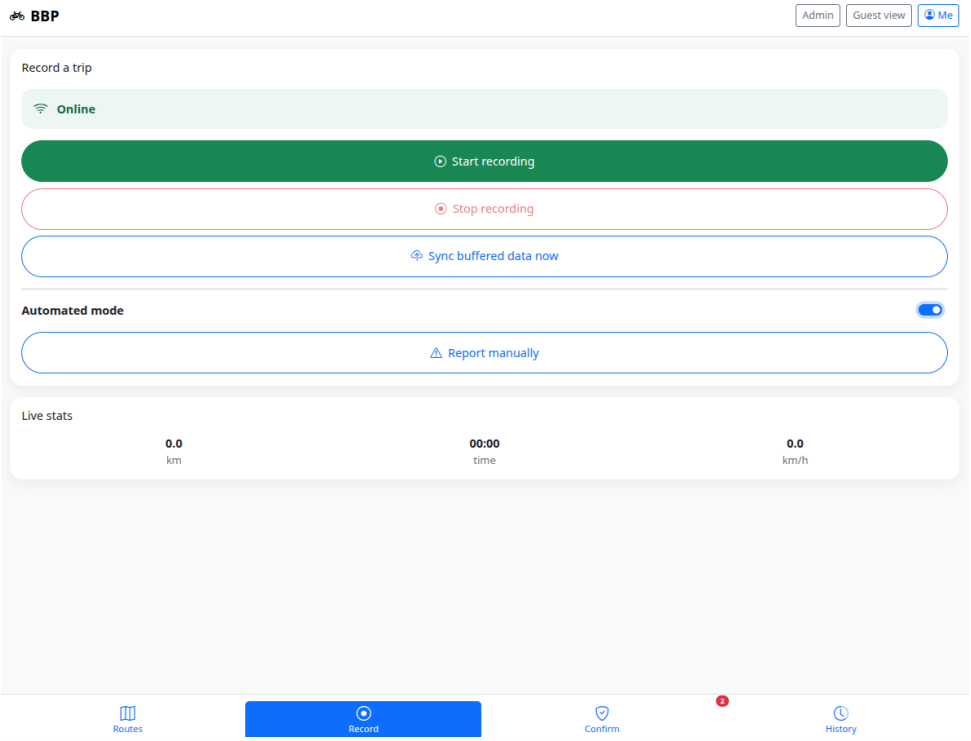



Figure 15: Record Bike Path

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

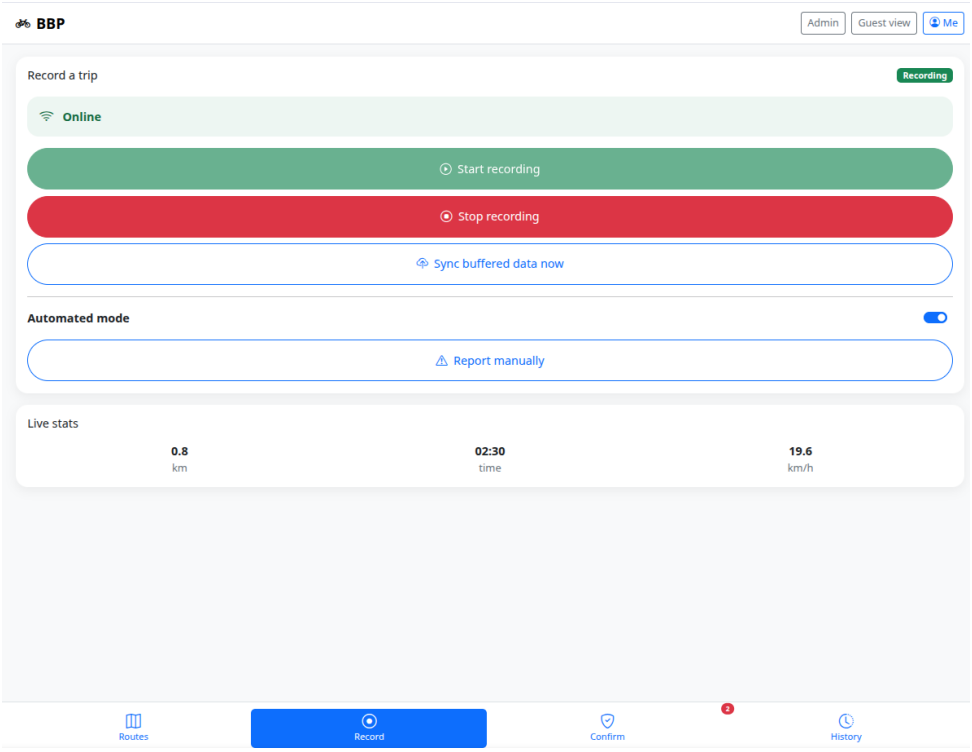



Figure 16: Record Bike Path Stats

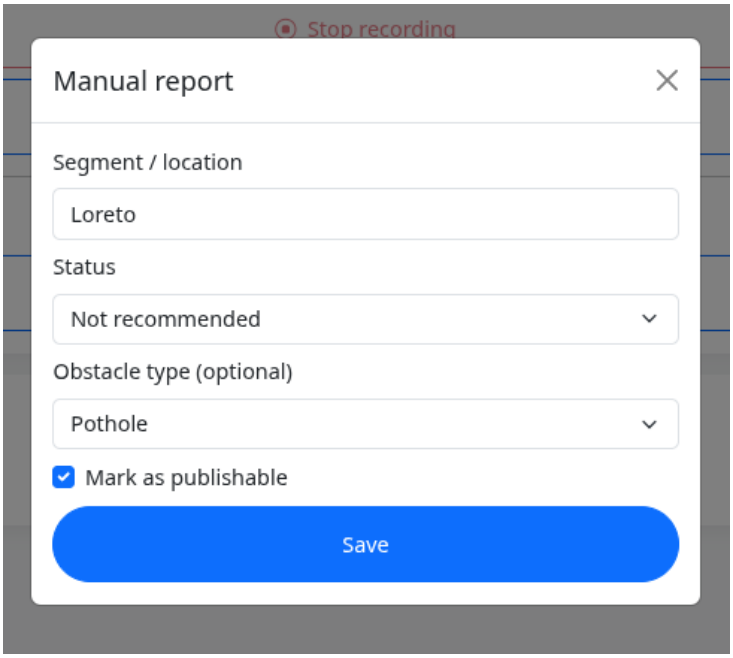



Figure 17: Manual report

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

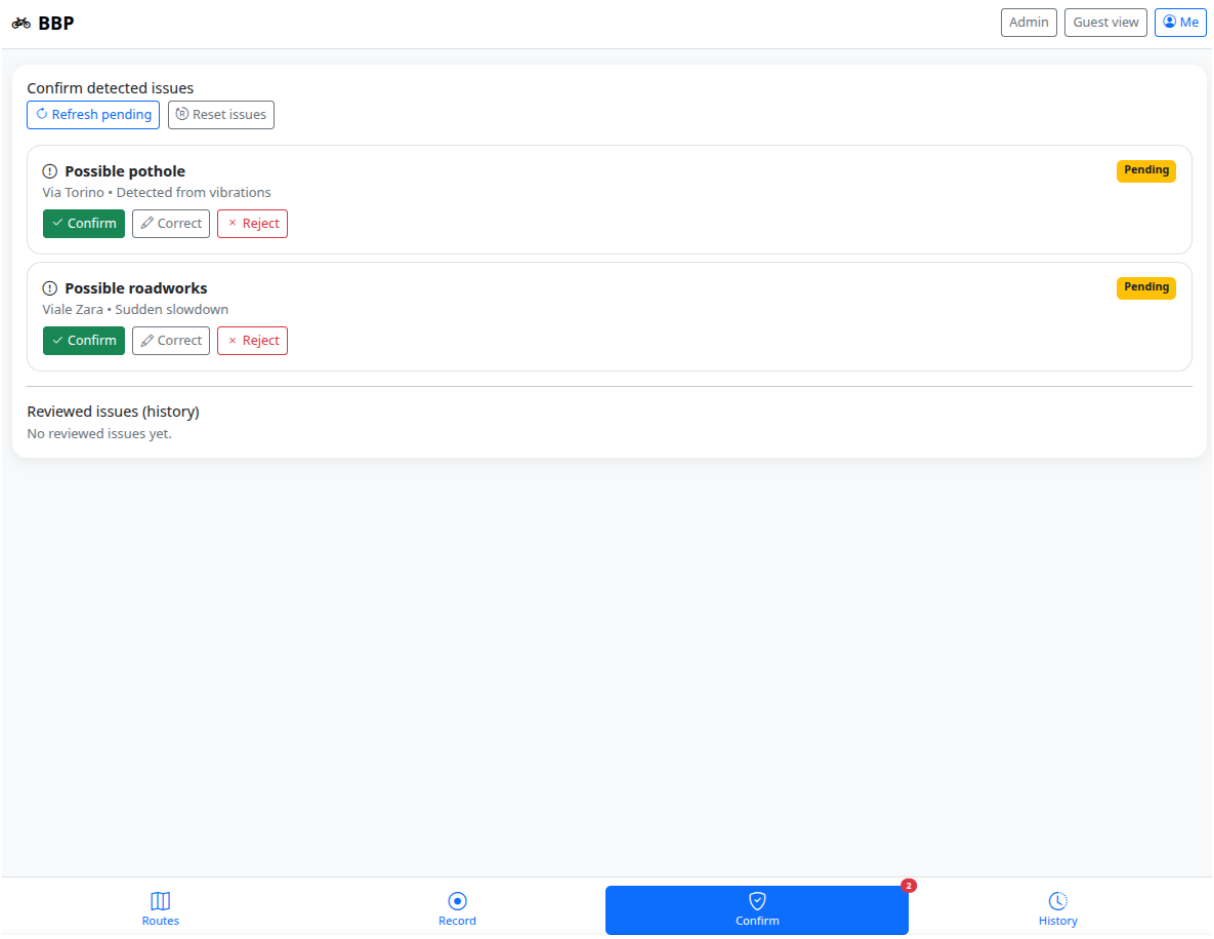



Figure 18: Detected issues status

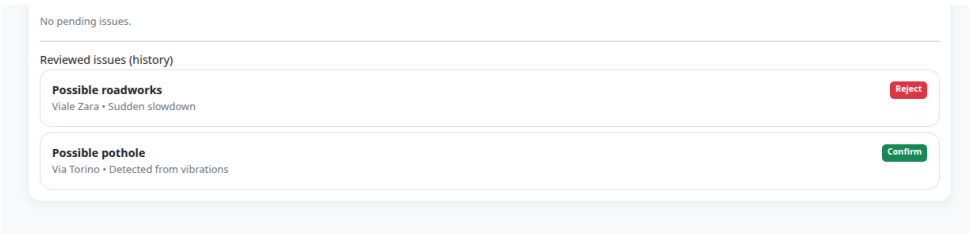



Figure 19: Review past detected issues

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

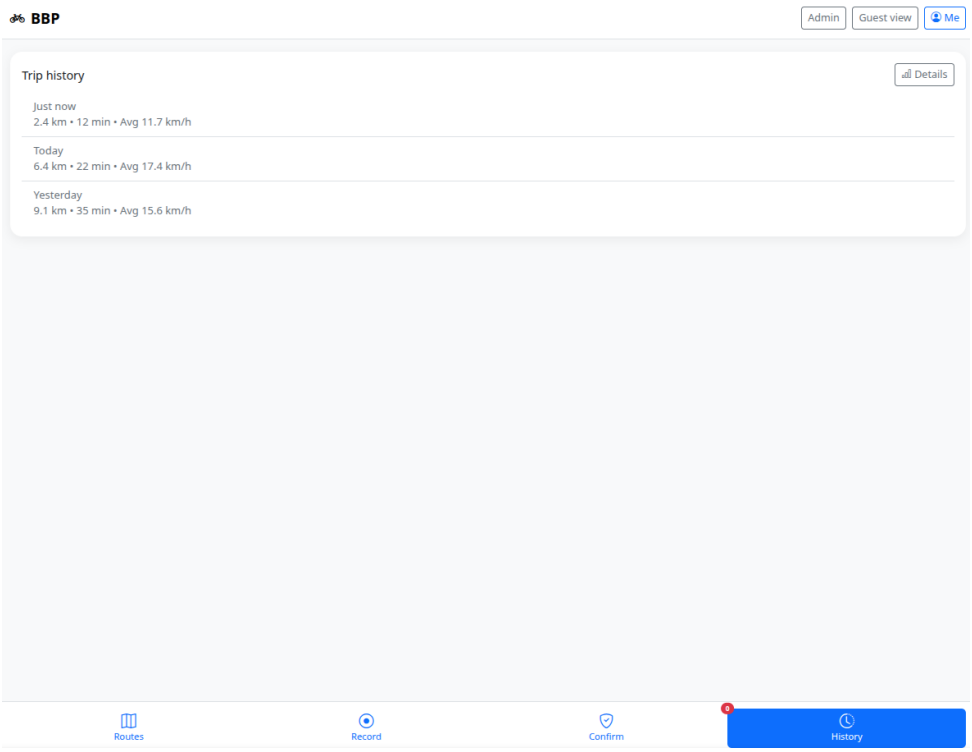



Figure 20: Trip history

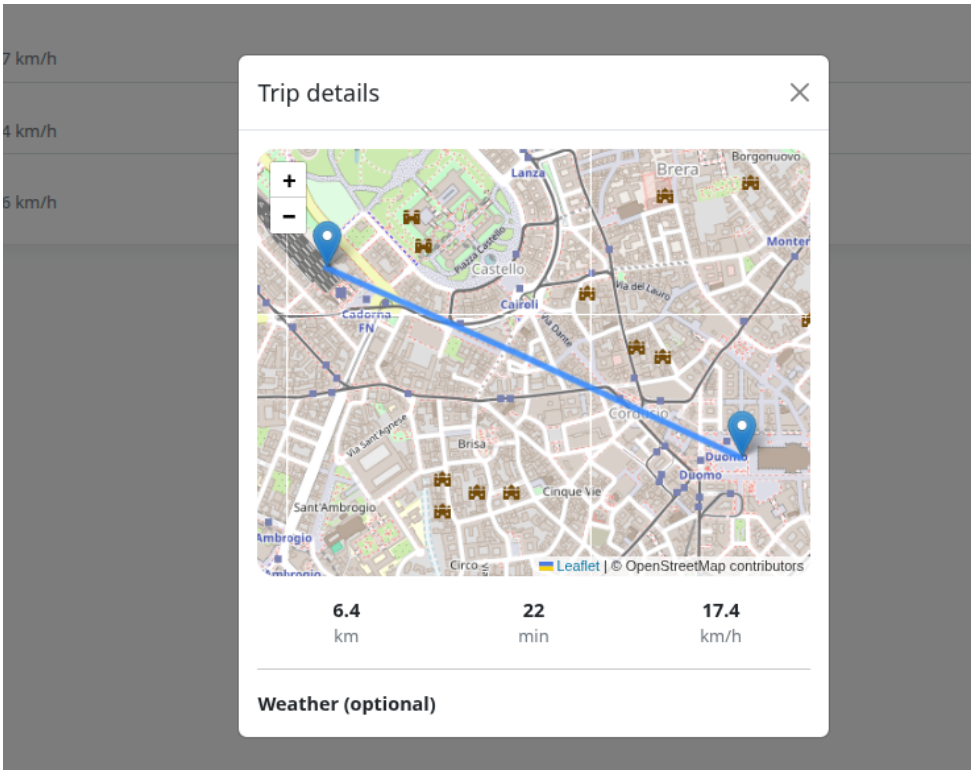



Figure 21: Trip history details

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 3.4 Manual Path

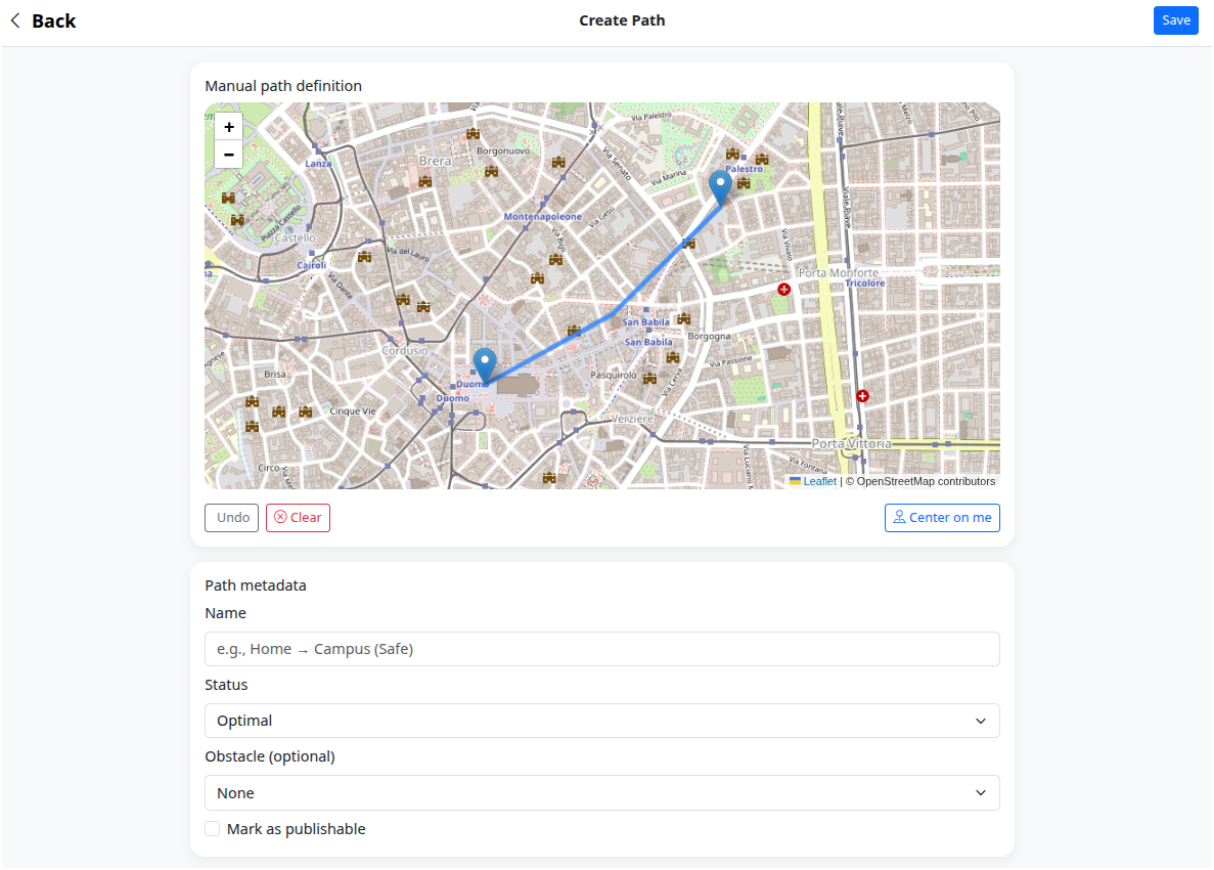



Figure 22: Create manual path

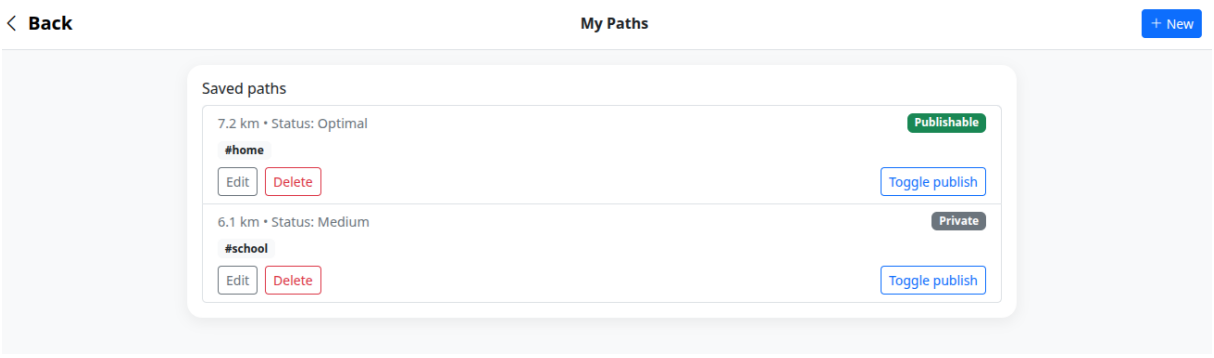



Figure 23: Manual paths history

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

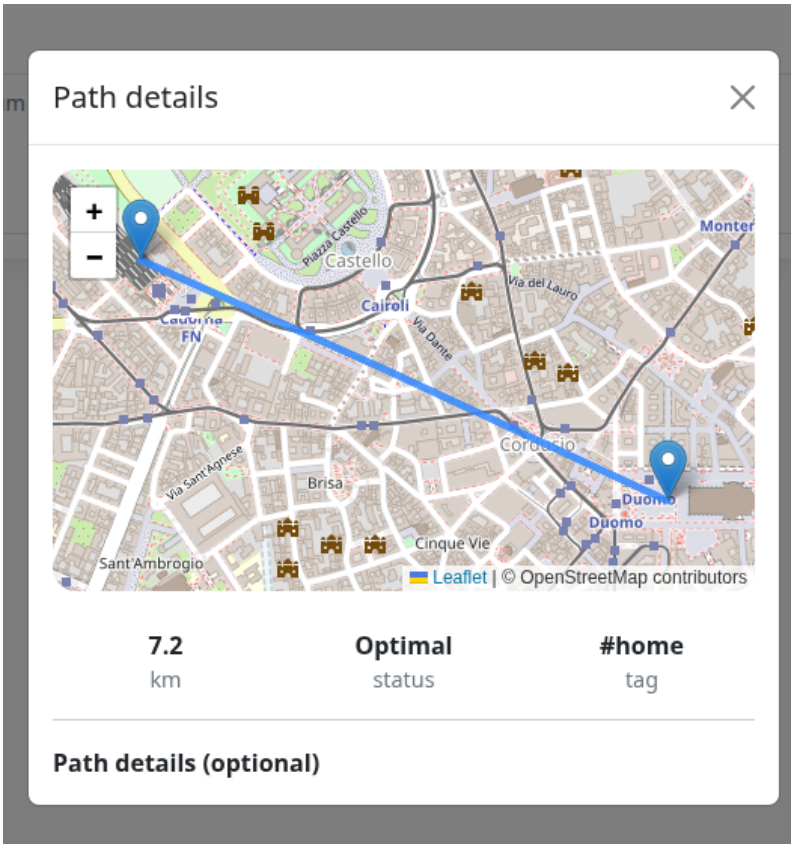



Figure 24: Manual paths history details

## 3.5 Profile

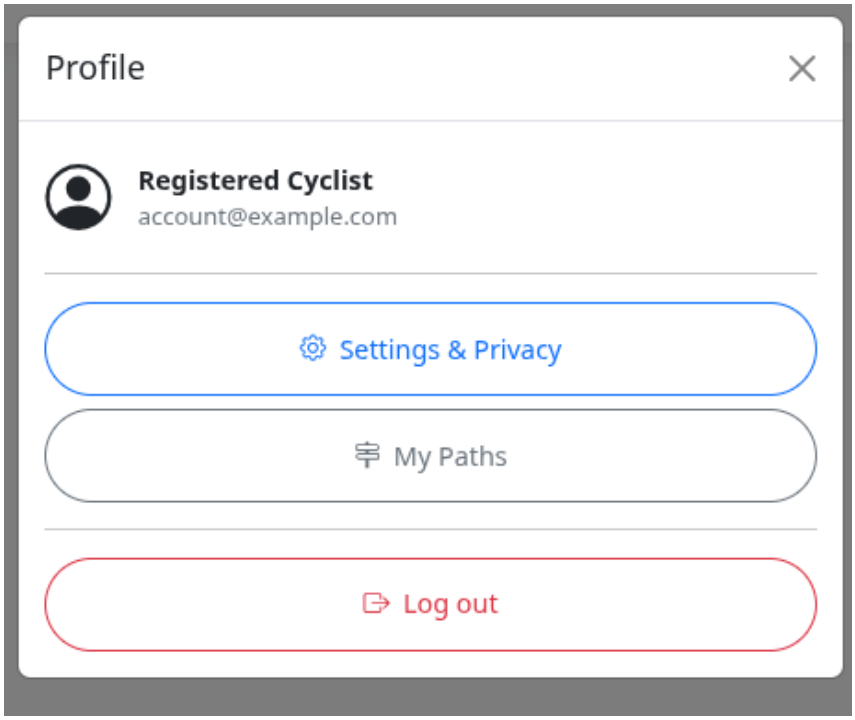



Figure 25: Registered user profile

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved

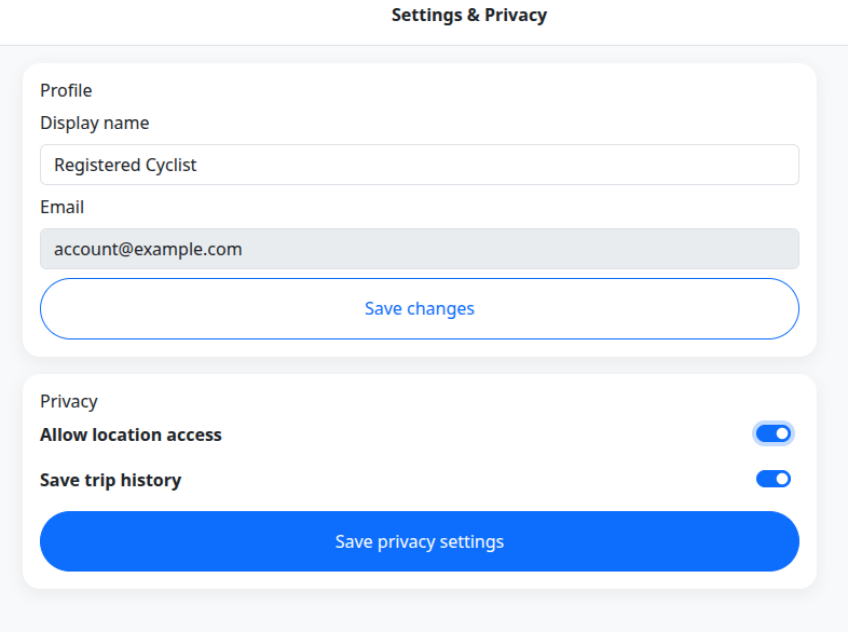



Figure 26: Registered user setting

## 3.6 Feedback

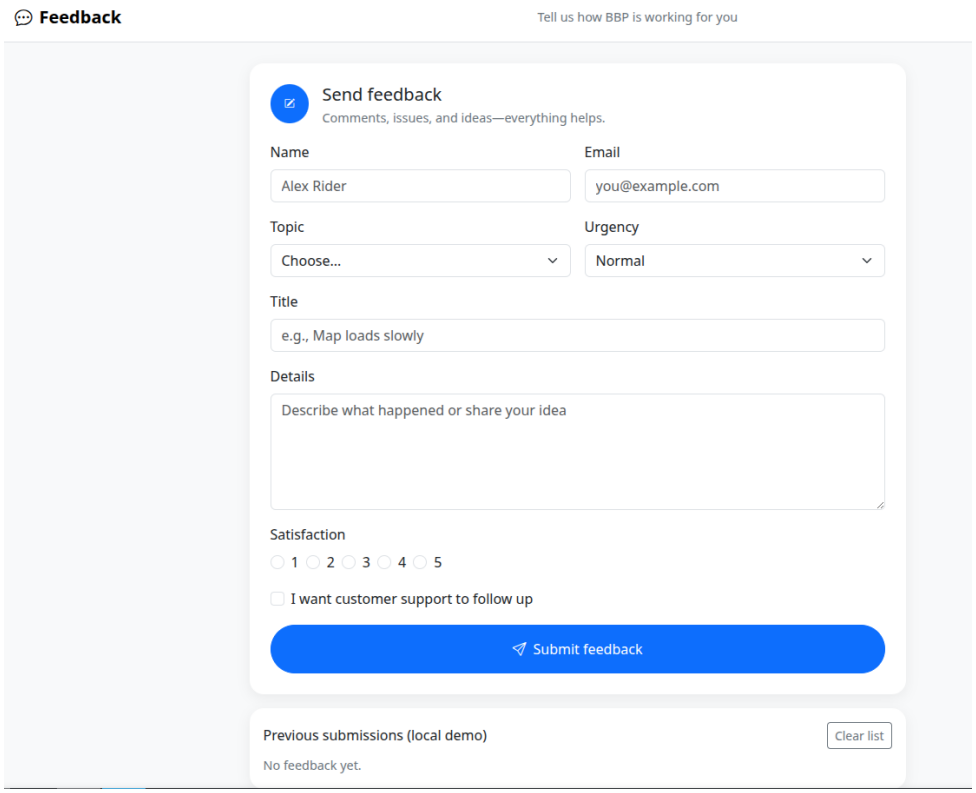



Figure 27: Registered user feedback

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 3.7 Administrative Overview

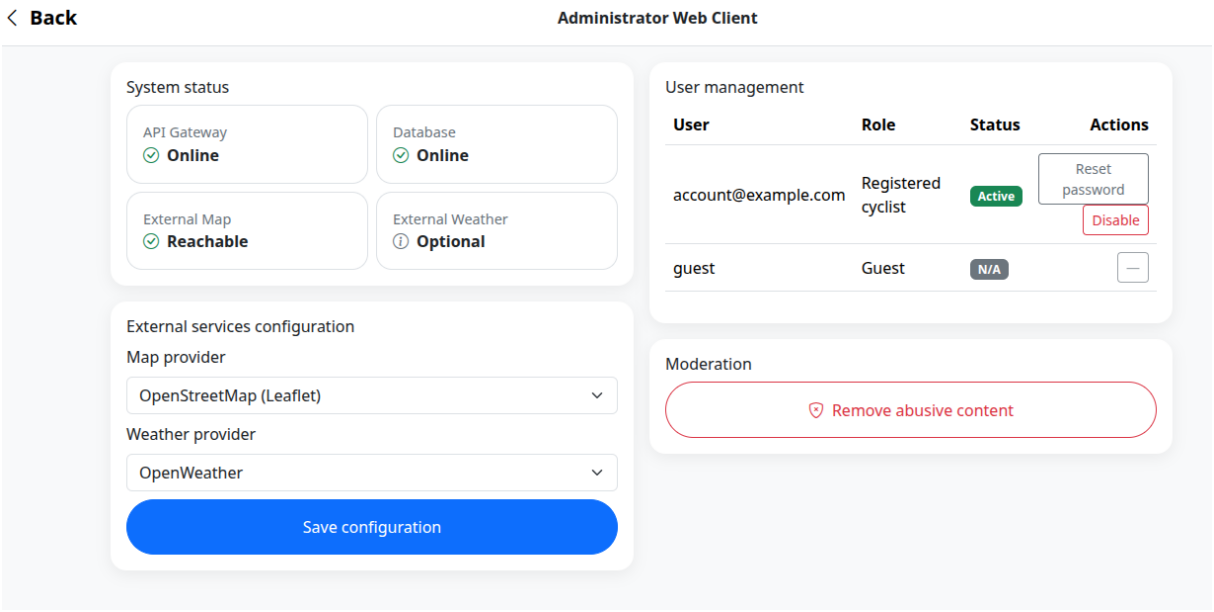



Figure 28: Administrative overview

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 4 Requirements Traceability

This section maps the functional requirements defined in the RASD to the main design components presented in Section 2.2 (Component View). The purpose is to show that every requirement is realized by one or more components in the Presentation Layer, Application Layer, and Data Layer. Unless explicitly stated, all persistence is handled through the **Database**, and all client–server communication is mediated by the **API Gateway**. :contentReference[oaicite:1]index=1

## 4.1 User Registration and Profile Management

| Requirement (R) | Design Components |
|---|---|
| **R1: User Registration** (Allow cyclists to create an account and access protected features) | Mobile Client, API Gateway, Authentication Service, Database |
| **R2: User Login/Logout** (Enable session creation and termination for registered cyclists) | Mobile Client, API Gateway, Authentication Service, Database |
| **R3: Profile Management** (Allow cyclists to view and update profile data) | Mobile Client, API Gateway, User Profile Service, Database |
| **R4: Privacy and Default Publication Preferences** (Allow cyclists to set default publishability settings) | Mobile Client, API Gateway, User Profile Service, Database |

Table 2: Traceability for registration and profile requirements.

## 4.2 Trip Recording and History

| Requirement (R) | Design Components |
|---|---|
| **R5: Trip Recording** (Allow cyclists to start and stop recording and collect GPS samples) | Mobile Client, API Gateway, Trip Service, Database |
| **R6: Offline Buffering** (Buffer collected samples when connectivity is not available and upload later) | Mobile Client, API Gateway, Trip Service, Database |
| **R7: Trip Statistics** (Compute and return distance, duration, and average speed) | Trip Service, API Gateway, Database, Mobile Client |
| **R8: Trip History** (Store and browse recorded trips and related statistics) | Mobile Client, API Gateway, Trip Service, Database |

Table 3: Traceability for trip recording and history requirements.

## 4.3 Weather Enrichment

| Requirement (R) | Design Components |
|---|---|
| **R9: Weather Enrichment** (Enrich trips with meteorological data when the provider is available) | Trip Service, Weather Adapter, Database |

Table 4: Traceability for weather enrichment requirements.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

### 4.4 Manual Path and Obstacle Contribution

| Requirement (R) | Design Components |
|---|---|
| **R10: Manual Path Definition** (Allow cyclists to define a path as connected segments) | Mobile Client, API Gateway, Path Service, Database |
| **R11: Path Status Assignment** (Allow cyclists to assign a status to paths or segments) | Mobile Client, API Gateway, Path Service, Database |
| **R12: Manual Obstacle Reporting** (Allow cyclists to report obstacles on segments) | Mobile Client, API Gateway, Path Service, Database |
| **R13: Publishable Information** (Allow cyclists to mark contributed information as publishable) | Mobile Client, API Gateway, User Profile Service, Path Service, Database |

Table 5: Traceability for manual contribution requirements.

### 4.5 Automated Detection and Issue Review

| Requirement (R) | Design Components |
|---|---|
| **R14: Automated Mode Activation** (Enable sensor-based detection during recording) | Mobile Client, API Gateway, Issue Detection Service, Database |
| **R15: Detected Issues Generation** (Produce detected issues from motion sensor signals) | Issue Detection Service, Database |
| **R16: Issue Confirmation/Correction/Reject** (Require user validation before publication) | Mobile Client, API Gateway, Issue Review Service, Database |
| **R17: Conversion to Obstacles** (Convert confirmed issues to obstacles and apply publishability rules) | Issue Review Service, Path Service, User Profile Service, Database |

Table 6: Traceability for automated detection and issue review requirements.

### 4.6 Route Query and Map Visualization

| Requirement (R) | Design Components |
|---|---|
| **R18: Route Query Submission** (Allow users to submit origin–destination requests) | Web Client, API Gateway, Route Query Service |
| **R19: Candidate Paths Retrieval** (Retrieve candidates from publishable information) | Route Query Service, Path Service, Database |
| **R20: Scoring and Ranking** (Compute scores based on status and publishable obstacles) | Route Query Service, Path Service, Database |
| **R21: Map Visualization** (Display candidate paths and obstacles on a map) | Web Client, API Gateway, Route Query Service, Map Adapter |
| **R22: Guest Access** (Allow non-registered users to query routes and view publishable information only) | Web Client, API Gateway, Route Query Service, Database |

Table 7: Traceability for route query and visualization requirements.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

## 4.7 Administration

| Requirement (R) | Design Components |
|---|---|
| **R23: Account Management** (Allow administrators to manage user accounts) | Administrator Web Client, API Gateway, Administration Service, Authentication Service, Database |
| **R24: External Services Configuration** (Configure connections to external providers) | Administrator Web Client, API Gateway, Administration Service, Database |
| **R25: Moderation** (Remove clearly incorrect publishable information when necessary) | Administrator Web Client, API Gateway, Administration Service, Path Service, Database |

Table 8: Traceability for administration requirements.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 5 Implementation, Integration and Test Plan

This section outlines the strategy to implement, integrate, and validate the BBP platform in an incremental and test-driven manner. We follow a bottom-up approach: we first implement and verify foundational components (data layer and core backend services), then progressively integrate external adapters and client applications, and finally perform end-to-end validation and non-functional testing.

Unless explicitly stated, all persistence is handled through the **Database**, and all client–server communication is mediated by the **API Gateway**.

## 5.1 Implementation Plan

### 5.1.1 Phase 1 – Data Layer and Shared Foundations

- **Database schema and migrations.** Design and implement the relational schema supporting users, trips (including GPS samples), paths/segments, obstacles, detected issues, and route queries. Add constraints, indices for query-critical attributes (e.g., segment identifiers, publishability flags), and migration scripts.

- **Security primitives in the data layer.** Store secrets only as salted hashes; enforce least-privilege DB accounts and prepared statements/ORM usage to reduce injection risks. Enable backup procedures.

- **Shared libraries.** Define common DTOs, error models, and validation utilities to keep service contracts consistent (e.g., input validation, pagination, timestamps).

### 5.1.2 Phase 2 – Core Backend Services

- **API Gateway.** Implement routing to backend services, authentication/authorization middleware, rate limiting, request validation, and consistent error handling.

- **Authentication Service.** Implement registration/login/logout, token/session issuance, and role-based authorization hooks required by the gateway and protected endpoints.

- **User Profile Service.** Implement profile CRUD and privacy/publication preferences, and expose APIs to other services for ownership/visibility checks.

### 5.1.3 Phase 3 – Trip and Contribution Domain

- **Trip Service.** Implement start/stop trip recording APIs, trip reconstruction storage, and statistics computation (distance, duration, average speed). Support ingestion of buffered trip points from the client.

- **Path Service.** Implement manual path creation/editing, segment status assignment, obstacle annotation, and visibility (publishable vs. private) enforcement.

- **Issue Detection Service.** Implement the pipeline that transforms motion sensor signals into candidate detected issues associated with trips/segments (initially with a baseline detection approach, then refined).

- **Issue Review Service.** Implement review workflows (confirm/correct/reject) and conversion of confirmed issues into obstacles, applying the publishability rules.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

#### 5.1.4 Phase 4 – Route Query and External Adapters

- **Route Query Service.** Implement origin–destination query submission, candidate retrieval from the publishable inventory, scoring/ranking logic, and results formatting for map visualization.
- **Map Adapter.** Provide an abstraction over the external map provider (tiles, geocoding, routing overlays), keeping provider-specific details isolated.
- **Weather Adapter.** Implement retrieval of meteorological data for a trip time window and bounding box, with error handling and fallback when the external service is unavailable.

#### 5.1.5 Phase 5 – Administration and Clients

- **Administration Service.** Implement moderation endpoints (remove abusive/incorrect publishable info), account management hooks, and configuration management for external services (map/weather).
- **Mobile Client.** Implement trip recording UI, offline buffering, issue review UI, and profile features.
- **Web Client.** Implement route query UI and map visualization, including guest access (publishable-only view).
- **Admin Web Client.** Implement monitoring, moderation, and configuration workflows.

#### 5.1.6 Phase 6 – Hardening and Operational Readiness

- **Observability.** Centralized logging, metrics, tracing, and alerting for gateway/services/DB.
- **Performance tuning.** Identify hot paths (candidate retrieval, scoring, map queries) and optimize queries/indexes.
- **Security review.** Threat modeling and vulnerability scanning; finalize rate limits and input validation.

### 5.2 Integration Plan

The integration plan follows staged integration, validating each stage with automated tests before moving to the next.

#### 5.2.1 Stage 1 – Database + Repository/DAO Layer

- Validate schema constraints, migrations, and CRUD correctness with seed data.
- Verify backup/restore procedures in a staging-like environment.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

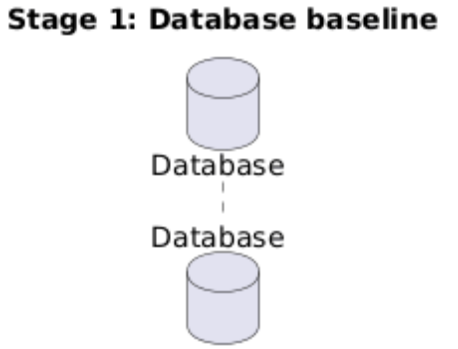



Figure 29: Stage 1 Database baseline

### 5.2.2 Stage 2 – Core Backend (Gateway + Auth + Profile)

- Integrate API Gateway with Authentication Service (token/session validation) and Profile Service (ownership/visibility lookups).
- Stabilize a consistent API error model and authentication flows used by all subsequent components.

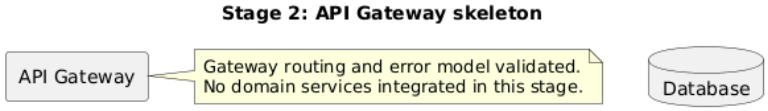



Figure 30: Stage 2 API Gateway skeleton

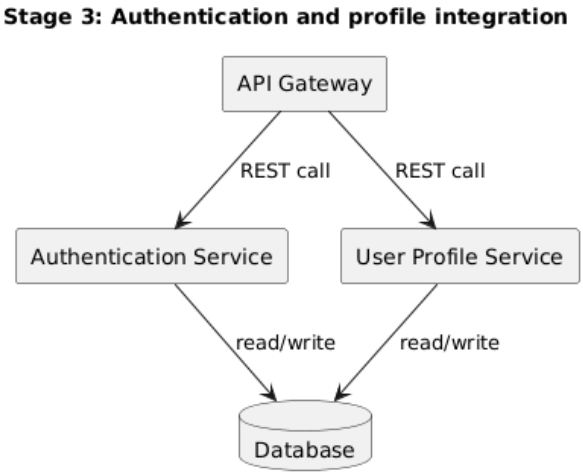



Figure 31: Stage 3 Authentication and profile core

### 5.2.3 Stage 3 – Domain Services with Database

- Integrate Trip Service with DB: trip lifecycle, point ingestion, and statistics persistence.
- Integrate Path Service with DB: path/segment CRUD, obstacle CRUD, publishability enforcement.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

- Integrate Issue Detection and Issue Review services with Trip/Path services and DB.

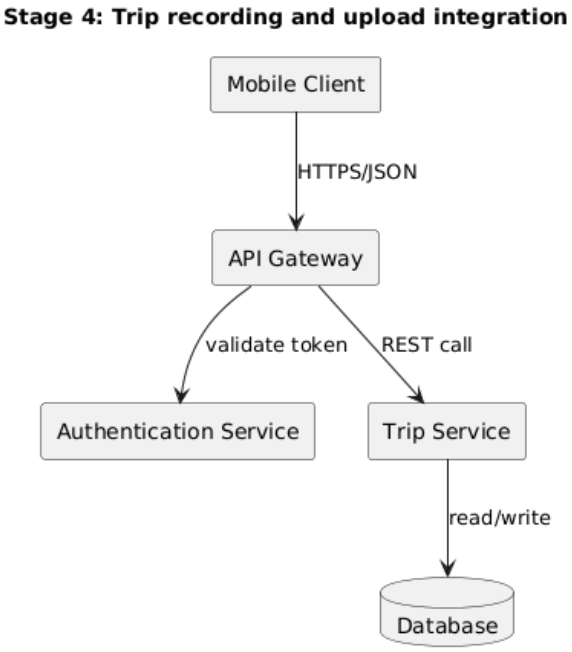



Figure 32: Stage 4 Trip recording and upload

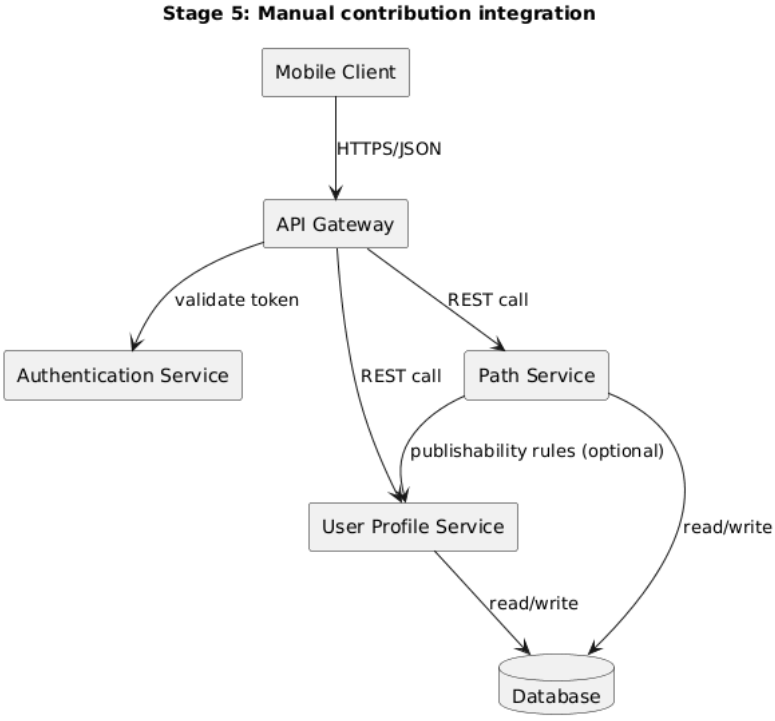



Figure 33: Stage 5 Manual path and obstacle contribution

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

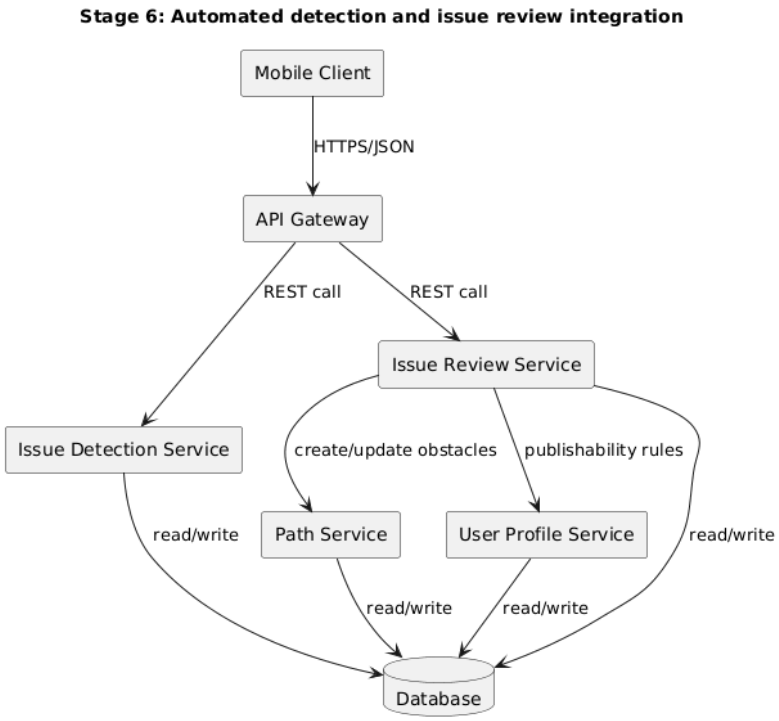



Figure 34: Stage 6 Automated detection and issue review

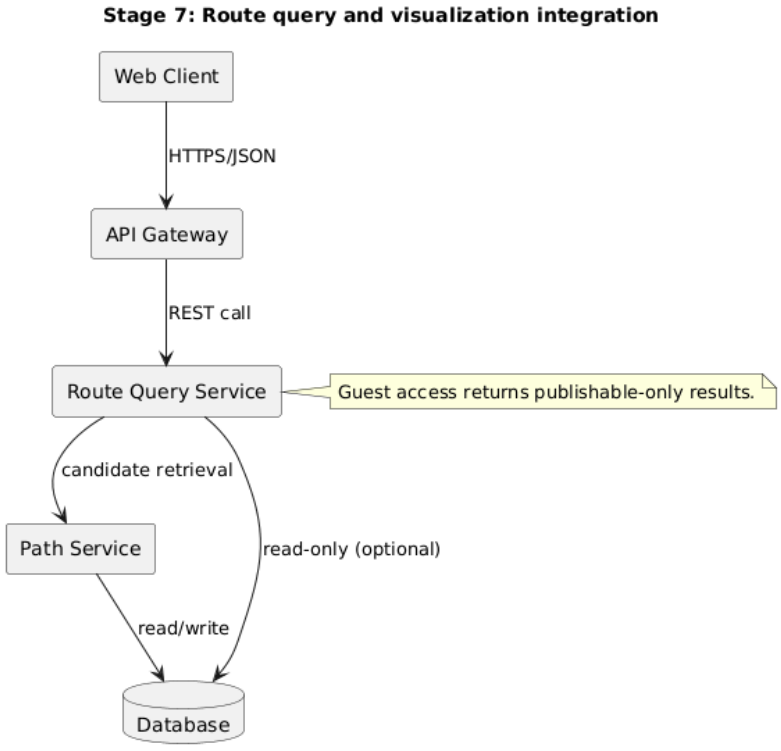



Figure 35: Stage 7 Route query and visualization (guest access)

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

### 5.2.4 Stage 4 – External Services via Adapters

- Integrate Weather Adapter behind the Trip Service enrichment workflow, using mocks first, then a real provider in staging.

- Integrate Map Adapter behind the Web Client visualization and Route Query Service output formatting.

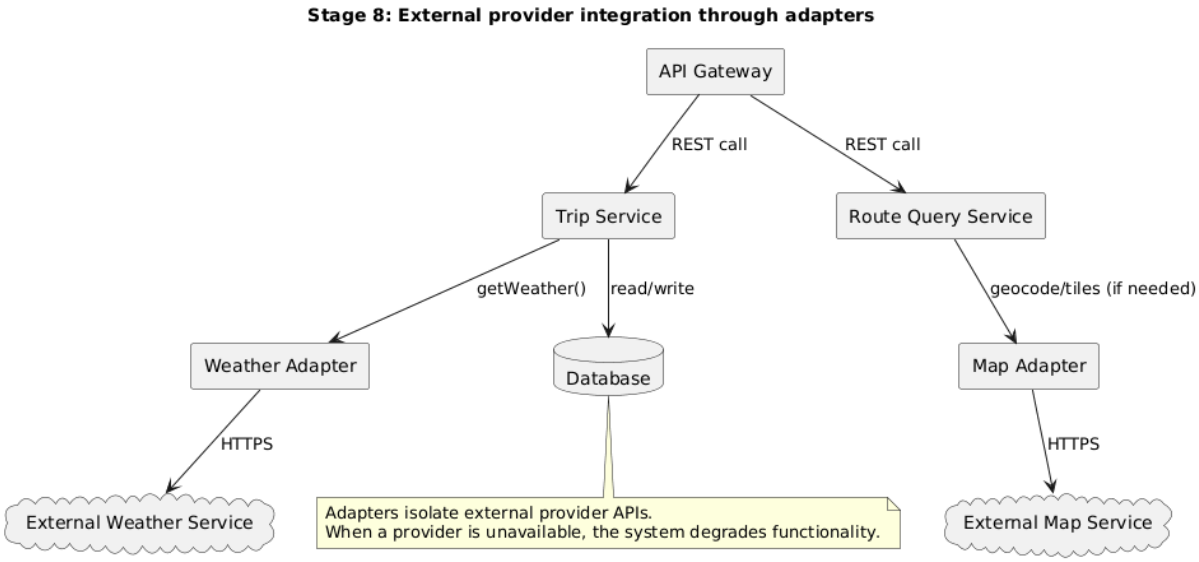



Figure 36: Stage 8 External providers via adapters (map + weather)

### 5.2.5 Stage 5 – End-to-End Client Integration

- Integrate Mobile Client with Gateway for registration/login, trip recording (including offline upload), issue review, and profile management.

- Integrate Web Client with Gateway for route queries, map visualization, and guest access behavior.

- Integrate Admin Web Client with Administration Service for moderation and configuration workflows.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

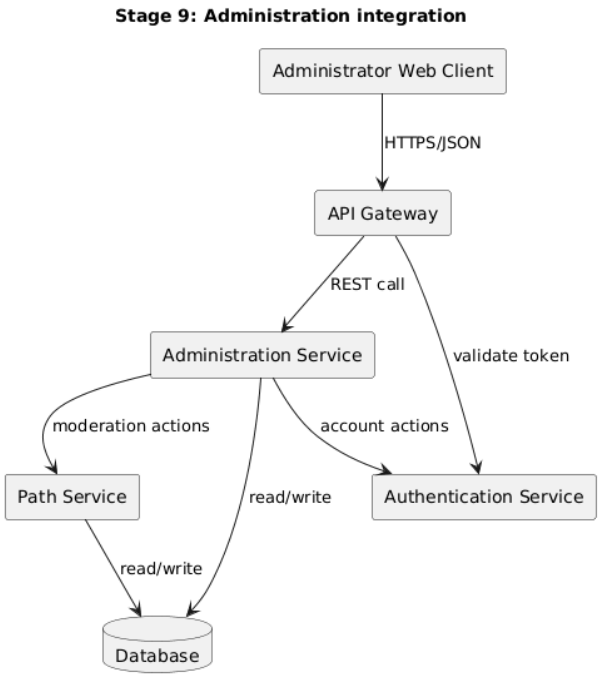



Figure 37: Stage 9 Administration

After each stage, regression tests are executed to ensure new integrations do not break previously validated features.

## 5.3 Test Plan

Testing is continuous and layered, combining automated unit/integration suites with scenario-based system tests and non-functional testing.

### 5.3.1 Unit Testing

- **Backend services:** validate business logic (visibility rules, ownership checks, scoring function, issue lifecycle transitions) with mocked DB/adapters.
- **Adapters:** parse/transform provider responses and handle error cases (timeouts, invalid payloads).
- **Clients:** validate local logic (offline buffer policies, input validation, UI state transitions).

### 5.3.2 Integration Testing

- **Service–DB tests:** verify schema assumptions, transactional behavior, and query performance for trip/path/issue workflows.
- **Service–service tests:** gateway-to-service routing, auth propagation, and cross-service invariants (e.g., confirmed issues always map to stored obstacles).
- **Adapter contract tests:** run against stub servers and (when possible) staging endpoints to detect provider API changes early.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

### 5.3.3 System Testing (End-to-End)

Execute end-to-end scenarios derived from the main use cases:

- Registration/login and profile update.
- Record trip (online and offline buffering) → stop trip → stats and history.
- Weather enrichment on trip closure, with graceful fallback when unavailable.
- Manual path definition, status assignment, obstacle reporting, and publication toggling.
- Automated detection during trip → review phase (confirm/correct/reject) → obstacle creation.
- Route query as guest vs. registered user, map visualization of candidate paths and obstacles.
- Admin moderation of abusive/incorrect publishable information.

### 5.3.4 Acceptance Testing

- Validate stakeholder-level workflows using realistic data and devices/browsers.
- Perform alpha tests internally (team-driven) and beta tests with external users when feasible.

### 5.3.5 Regression Testing

- Maintain an automated regression suite for critical workflows (auth, trip close, route query, publishability).
- Run the suite after every integration stage and before each release candidate.

### 5.3.6 Performance, Load, and Stress Testing

- **Load tests:** simulate concurrent route queries and trip history browsing; verify latency targets and DB utilization.
- **Stress tests:** push beyond expected peak to observe graceful degradation (e.g., rate limiting) rather than total failure.
- **Soak tests:** sustained load to detect memory leaks and resource exhaustion.

### 5.3.7 Security Testing

- Verify authentication and authorization rules (publishable-only access for guests, ownership constraints for users).
- Test common vulnerabilities (SQL injection, XSS, CSRF where applicable), enforce TLS, and review dependency vulnerabilities.

### 5.3.8 Deployment Testing

- Validate repeatable deployment in a production-like environment (container orchestration, secrets management, migrations).
- Verify compatibility across supported mobile devices and modern browsers for the Web Client.
- Validate backup/restore and rollback procedures.

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 6 Effort Spent

This section includes information about how much effort each group member spent in working on this document.

- *Hamidreza Khodadadi*

| Chapter | Hours |
|---|---|
| 1 | 2 |
| 2 | 3 |
| 3 | 10 |
| 4 | 4 |
| 5 | 3 |

- *Mohammad Rashid Khan Mofrad*

| Chapter | Hours |
|---|---|
| 1 | 2 |
| 2 | 14 |
| 3 | 2 |
| 4 | 3 |
| 5 | 6 |

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 

# 7 References

- Overleaf as LATEX editor
- GitHub for project versioning
- PlantUML for UML diagrams
- Alloy for formal analysis

Copyright © 2025, Mohammad RashidKhan and Hamidreza Khodadadi – All rights reserved 